VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 9: Layered Testbench Architecture

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 9 Overview ... 3

Part 1: Direct Testbench Versus Layered Testbench ... 5
- 1.1 What Every Testbench Must Do ... 5
- 1.2 The Direct Testbench ... 5
- 1.3 The Layered Testbench ... 8
- 1.4 Direct Versus Layered: Comparison ... 11
- 1.5 Why Industry Uses the Layered Architecture ... 12

Part 2: The Project at a Glance and Its Foundation Files ... 14
- 2.1 What Is Being Verified ... 14
- 2.2 Who Creates Whom, and Who Shares What ... 15
- 2.3 The DUT Files ... 16
- 2.4 The Interface: alu_if.sv ... 18
- 2.5 The Transaction: alu_txn.sv ... 21

Part 3: The Stimulus Side: Generator and Driver ... 25
- 3.1 The Generator ... 25
- 3.2 The Driver ... 30

Part 4: The Checking Side: Monitor and Scoreboard ... 36
- 4.1 The Monitor ... 36
- 4.2 The Scoreboard ... 41

Part 5: Environment, Test, Package and Testbench Top ... 48
- 5.1 The Environment ... 48
- 5.2 The Test ... 52
- 5.3 The Testbench Package ... 54
- 5.4 The Testbench Top: tb_top.sv ... 56
- 5.5 Compiling and Running the Project ... 59

Part 6: Execution from Start to Finish ... 61
- 6.1 The Complete Communication Flow ... 61
- 6.2 Simulation Execution Order ... 62
- 6.3 Cycle-by-Cycle Timeline of One Transaction ... 64
- 6.4 A Directed Test, with Its Exact Log ... 65
- 6.5 The Random Test ... 69
- 6.6 Adding a Test Without Touching the Components ... 69
- 6.7 A Seeded Bug: What a Failure Looks Like ... 72
- 6.8 The Same Structure for a Different DUT: a Synchronous FIFO ... 73
- 6.9 The Classic Design Questions, Answered in One Place ... 75

Part 7: Module 9 Revision and Practice ... 78
- 7.1 Revision Summary ... 78
- 7.2 Interview Questions and Answers ... 80
- 7.3 Output-Prediction Questions ... 83
- 7.4 Find-the-Error and Debugging Questions ... 87
- 7.5 Coding Exercises with Solutions ... 91
- 7.6 Concepts You Must Understand Before Moving Forward ... 96
- 7.7 What Comes Next ... 97

Module 9 Overview

Modules 1 to 8 taught the individual pieces of a class-based testbench: data types, arrays, tasks and functions, classes, randomization, transaction classes, mailboxes and events, interfaces and clocking blocks. This module puts all of them together. By the end of it you will have a complete, working **layered testbench** for the 8-bit ALU from Module 1, and you will be able to explain every file, every connection and every design decision in it.

This is the module interviewers draw on most for "architecture" questions: "Draw a testbench.", "What does the monitor do?", "How does your test end?", "Why does the scoreboard not take its expected values from the generator?". Each of these has a definite answer, and each answer is in this module.

What this module contains

Part	Content
1	Direct testbench versus layered testbench: what each is, advantages, disadvantages, why industry uses the layered form
2	The project at a glance, and its foundation files: DUT, interface with modports and clocking blocks, transaction class
3	The stimulus side: generator and driver
4	The checking side: monitor and scoreboard
5	Environment, test, testbench package and testbench top; how to compile and run
6	Execution from start to finish: communication flow, cycle-by-cycle timeline, logs, end of test, new tests without touching components, a seeded bug, and the same structure applied to a FIFO
7	Revision summary, interview questions, output prediction, debugging, coding exercises, checklist

What you need from earlier modules

Topic	Module
`logic` versus `bit`, enums, `!==`, expression width	1
Queues	2
Tasks, functions, `automatic`	3
Classes, handles, `new`, inheritance, virtual methods, shallow and deep copy	4
`rand`, constraints, `randomize()`	5
Transaction class with `print`, `copy`, `compare`	6
Mailboxes, events, `fork` ... `join_any`, `disable fork`	7
Interfaces, modports, virtual interfaces, clocking blocks	8

Each of these is recalled in one or two sentences at the point where it is used, so that the module can be read on its own.

The project files

File	Kind	Contents
`alu_pkg.sv`	Package	The opcode enum `alu_op_e` (from Module 1)

File	Kind	Contents
`alu.sv`	Module	The DUT (from Module 1, unchanged)
`alu_if.sv`	Interface	DUT signals, clocking blocks `drv_cb` and `mon_cb`, modports
`alu_txn.sv`	Class	The transaction
`alu_generator.sv`	Class	Creates and randomizes transactions
`alu_driver.sv`	Class	Turns transactions into pin activity
`alu_monitor.sv`	Class	Turns pin activity back into transactions
`alu_scoreboard.sv`	Class	Predicts the expected result and compares
`alu_env.sv`	Class	Creates and connects the four components
`alu_test.sv`	Class	Base test: configures and runs the environment
`alu_arith_test.sv`	Classes	A derived test that only generates ADD and SUB
`alu_directed_test.sv`	Classes	A derived test with eight hand-written transactions
`alu_tb_pkg.sv`	Package	Includes all the class files
`tb_top.sv`	Module	Clock, reset, interface instance, DUT instance, starts the test

All of these are also saved in the `examples/module-09/` folder of the repository.

> **Note:** As in the other modules, the logs in this module were worked out by hand from the IEEE 1800 scheduling rules, because no simulator was available while writing. The directed test has a fully determined log, given in Section 6.4. The random tests print different operand values with every seed and simulator; their logs are labelled as samples. Run the project yourself and compare.

Part 1: Direct Testbench Versus Layered Testbench

1.1 What Every Testbench Must Do

In plain English

Whatever its style, a testbench has five jobs. Keep this list in mind, because the whole of this module is about *where* each job is done.

Job	Question it answers
1. Generate stimulus	What values and operations should be applied to the DUT?
2. Drive the stimulus	How are those values put onto the DUT pins, and when, relative to the clock?
3. Observe the DUT	What actually happened on the pins?
4. Check the response	Was the DUT output correct for the input it received?
5. Measure progress	Which scenarios have been exercised (coverage), and did the test pass?

A **direct testbench** does all five jobs in one place, usually one module with a few tasks. A **layered testbench** gives each job to a separate component and connects the components through well-defined channels.

The two levels of abstraction

Jobs 1 and 4 are about *what* is being done: "add 200 and 100; the answer must be 300". Jobs 2 and 3 are about *how* it appears on wires: "at the rising edge, `valid_in` is 1, `a` is `C8`, `b` is `64`, `op` is `000`; one cycle later `valid_out` is 1 and `result` is `12C`".

Level	Unit of work	Who works at this level
Transaction level	A transaction object: operands, opcode, result	Generator, scoreboard, test
Signal (pin) level	Values of individual signals at clock edges	Driver, monitor, interface, DUT

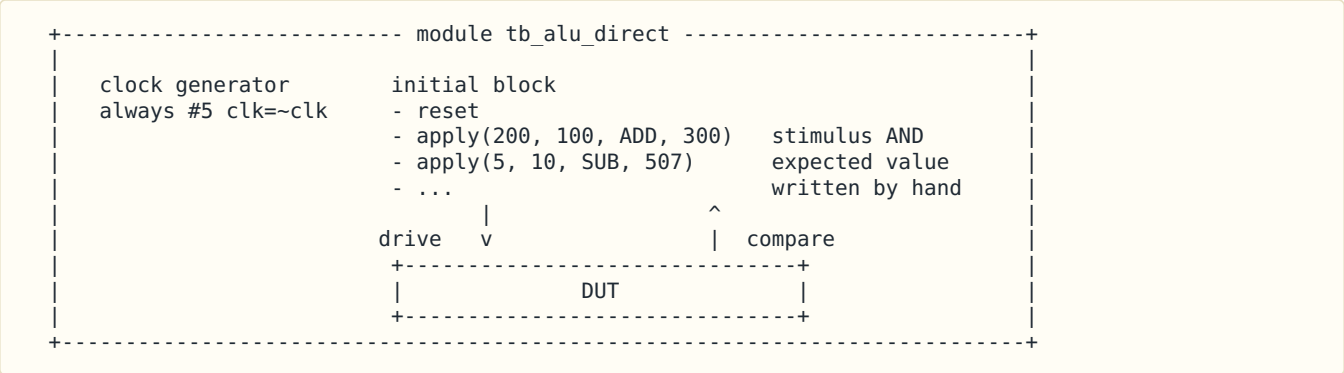
The driver converts from transaction level down to signal level. The monitor converts from signal level back up to transaction level. Everything else in the testbench never touches a signal. This separation is the central idea of a layered testbench.

1.2 The Direct Testbench

Definition

A direct (also called flat, linear or module-based) testbench is a single module that instantiates the DUT, generates the clock, and contains procedural code that assigns values straight to the DUT inputs and compares the DUT outputs against values the engineer worked out by hand. It is the Verilog style of verification.

Structure



Complete example

This is a slightly improved version of the directed testbench from Module 1. A task removes the repetition, and an error counter gives a final verdict. Compile it together with `alu_pkg.sv` and `alu.sv`.

tb_alu_direct.sv

```
module tb_alu_direct;
  import alu_pkg::*;

  logic        clk = 0;
  logic        rst_n;
  logic        valid_in;
  logic [7:0]  a, b;
  alu_op_e     op;
  logic        valid_out;
  logic [8:0]  result;
  int unsigned errors;

  alu dut (.*);

  always #5 clk = ~clk;

  task automatic apply(input logic [7:0] a_i,
                       input logic [7:0] b_i,
                       input alu_op_e    op_i,
                       input logic [8:0] exp_i);
    @(negedge clk);
    a        = a_i;
    b        = b_i;
    op       = op_i;
    valid_in = 1'b1;
    @(negedge clk);
    valid_in = 1'b0;
    if (valid_out !== 1'b1 || result !== exp_i) begin
      errors++;
      $display("[%0t] FAIL %s a=%0d b=%0d result=%0d expected=%0d",
               $time, op_i.name(), a_i, b_i, result, exp_i);
    end
    else
      $display("[%0t] PASS %s a=%0d b=%0d result=%0d",
               $time, op_i.name(), a_i, b_i, result);
  endtask

  initial begin
    rst_n    = 1'b0;
    valid_in = 1'b0;
    a        = '0;
    b        = '0;
    op       = ALU_ADD;
    repeat (2) @(negedge clk);
    rst_n = 1'b1;

    apply(8'd200, 8'd100, ALU_ADD, 9'd300);
```

```
    apply(8'd5,   8'd10,  ALU_SUB, 9'd507);
    apply(8'hF0,  8'h3C,  ALU_AND, 9'h030);
    apply(8'h81,  8'h00,  ALU_SHL, 9'h102);

    if (errors == 0) $display("TEST PASSED");
    else             $display("TEST FAILED with %0d errors", errors);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[40] PASS ALU_ADD a=200 b=100 result=300
[60] PASS ALU_SUB a=5 b=10 result=507
[80] PASS ALU_AND a=240 b=60 result=48
[100] PASS ALU_SHL a=129 b=0 result=258
TEST PASSED
```

Line by line

Code	Explanation
`alu dut (.*);`	Instantiates the DUT and connects each port to the testbench signal of the same name.
`always #5 clk = ~clk;`	Clock with a period of 10 time units. Rising edges at 5, 15, 25...; falling edges at 10, 20, 30...
`task automatic apply(...)`	One task does three of the five jobs: it drives the pins, waits, and checks. `automatic` gives each call its own copy of the arguments (Module 3).
`@(negedge clk);` then the four assignments	Stimulus is applied on the falling edge, half a cycle away from the rising edge at which the DUT samples. That avoids a race between testbench and DUT.
Second `@(negedge clk);`	By now one rising edge has passed, so the DUT has registered the result.
`valid_out !== 1'b1 || result !== exp_i`	The check, with `!==` so that X on an output is a failure (Module 1, Section 4.5).
`apply(8'd5, 8'd10, ALU_SUB, 9'd507);`	The engineer calculated the expected value by hand: 5 - 10 = -5, which in 9 bits is 512 - 5 = 507.
`if (errors == 0) ...`	Final verdict.

Execution flow

Time	What happens
0	Inputs get known values and reset is asserted.
20	Second falling edge: reset released. First `apply` call starts and waits for the next falling edge.
30	`apply` drives 200, 100, `ALU_ADD`, `valid_in` = 1.
35	Rising edge: the DUT stores 300 and sets `valid_out`.
40	`apply` removes `valid_in`, checks, prints. The second call starts.
50, 55, 60	The same three steps for the subtraction. And so on every 20 time units.

Advantages of a direct testbench

Advantage	Explanation
Quick to write	A few dozen lines; no classes, no interface, no mailboxes

Advantage	Explanation
Easy to read for a small DUT	The stimulus and the expected value sit on the same line
Easy to debug	One thread of execution, strictly sequential
Exactly repeatable	The same vectors every run
No knowledge of OOP needed	Usable by a designer for a quick sanity check of a block

Disadvantages of a direct testbench

Disadvantage	What it means in practice
Expected values are worked out by hand	The ALU has 8 operations and 65,536 operand pairs each: 524,288 cases. Nobody writes 524,288 `apply` lines, and a hand calculation can itself be wrong
Only the scenarios you thought of are tested	Bugs live in the cases nobody thought of
Stimulus, pin timing and checking are tangled together	The task `apply` knows the protocol (drive on negedge, wait one cycle) *and* the data *and* the check. A change to any one of them means editing the same code
A protocol change breaks every test	If the ALU gains a second pipeline stage, every place that assumes "result after one cycle" must be edited
No reuse	The code cannot be reused for another test, at the next level of integration (block inside a subsystem), or on the next project
Checking happens only where the test looks	Between two `apply` calls nobody watches the outputs. A spurious `valid_out` during idle cycles goes unnoticed
One sequential thread	Real designs have several interfaces active at the same time; a single thread cannot drive and watch them all
No measurement of what was covered	"Are we done?" has no objective answer
Does not scale to a team	One file, one author

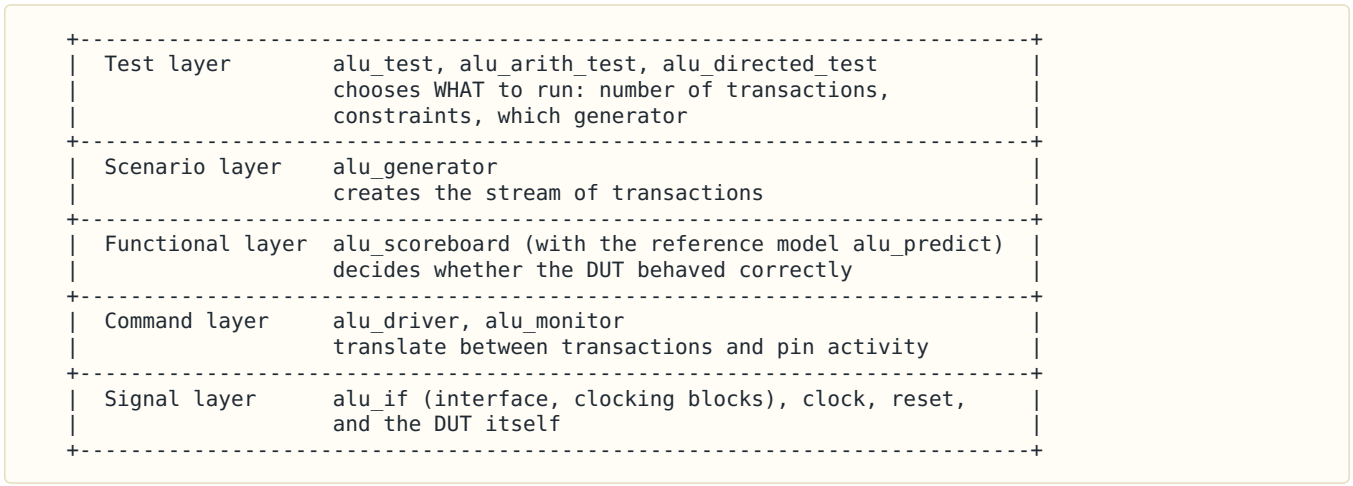
> **Key point:** The direct testbench is not wrong. It is the right tool for a 20-line module, for bring-up of a new block ("does it do anything at all?"), and for reproducing one specific failure. It stops being the right tool when the number of scenarios is too large to write by hand.

1.3 The Layered Testbench

Definition

A layered testbench is a testbench built from separate class-based components, each with one responsibility, arranged in layers of abstraction. The components exchange **transaction objects** through mailboxes, and only two of them (driver and monitor) touch the DUT signals, through a virtual interface. Stimulus is usually constrained-random, and checking is automatic.

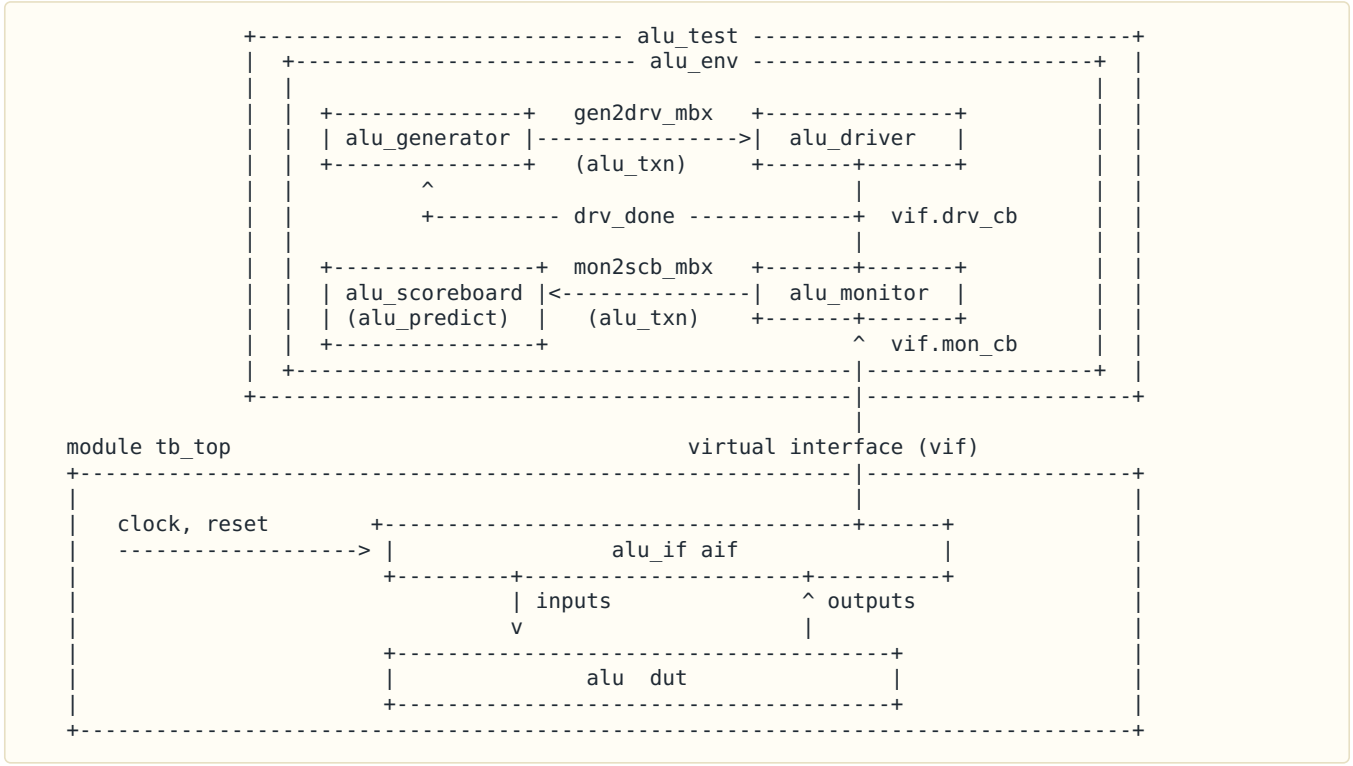
The layers



Reading from the top: each layer uses only the layer directly below it and knows nothing about the layers further down. The test does not know which clock edge the driver uses. The generator does not know that a signal called `valid_in` exists. The scoreboard does not know how the result reached it.

Layer	Works with	Knows about the protocol timing?	Changes when...
Test	Configuration, constraints	No	A new scenario is wanted
Scenario	Transactions	No	The kind or order of stimulus changes
Functional	Transactions	No	The DUT's intended function changes
Command	Transactions on one side, signals on the other	Yes	The pin-level protocol changes
Signal	Signals	It *is* the protocol	The DUT ports change

The components and the data flow



The two paths to remember, in the form interviewers ask for them:

```
Stimulus path :  Generator -> Mailbox -> Driver -> Interface -> DUT

Checking path :  DUT -> Interface -> Monitor -> Mailbox -> Scoreboard
```

Component	One-sentence job	Section
Transaction (`alu_txn`)	The unit of data that travels between components	2.5
Generator	Creates and randomizes transactions and sends them to the driver	3.1
Driver	Receives transactions and drives the DUT input pins at the right clock edges	3.2
Interface	Bundles the DUT signals and defines sampling and driving timing	2.4
Monitor	Watches the pins and rebuilds transactions from what it sees	4.1
Scoreboard	Predicts the expected result and compares it with the observed result	4.2
Environment	Creates the components, the mailboxes and the events, and connects them	5.1
Test	Configures the environment for one scenario and starts it	5.2
Testbench top	Static module: clock, reset, interface instance, DUT instance, starts the test	5.4

How the five jobs map onto components

Job	Direct testbench	Layered testbench
Generate stimulus	Literal values in the `initial` block	Generator plus constraints in the transaction class
Drive	Assignments inside the same task	Driver
Observe	Reading DUT outputs inside the same task	Monitor

Job	Direct testbench	Layered testbench
Check	`if` with a hand-computed value	Scoreboard with a reference model
Measure	Error counter	Scoreboard counters; functional coverage (Module 10)

Advantages of a layered testbench

Advantage	Explanation
Automatic checking of any stimulus	The reference model computes the expected value for whatever the generator produced, so thousands of random transactions are checked with no extra code
Finds unexpected bugs	Constrained-random stimulus reaches combinations nobody listed
Separation of concerns	A change in pin timing touches only the driver and monitor. A change in function touches only the reference model. A new scenario touches only a test
New tests are small	A test is a few lines that change constraints or counts (Section 6.6); the components are not edited
Reuse	The monitor and scoreboard can be reused unchanged when the ALU is embedded in a larger design, where the driver is replaced by real logic
Continuous observation	The monitor watches every clock edge, not only the moments when a test chooses to look
Parallelism	Each component runs as its own thread, which matches the concurrency of hardware
Team scalability	Different engineers own different components behind agreed interfaces (the transaction class and the mailboxes)
Path to UVM	UVM is this same architecture with standard base classes (Module 11)

Disadvantages of a layered testbench

Disadvantage	Explanation
Higher initial cost	Ten or more files have to exist before the first transaction runs
Needs more knowledge	OOP, randomization, IPC and virtual interfaces are all prerequisites
Harder to debug	Several threads, zero-time handshakes, handles and mailboxes; the cause of a failure can be in the testbench itself
The reference model is extra code that can be wrong	A bug in the model gives false failures, or, worse, the same misunderstanding in model and DUT gives a false pass
Random tests are not repeatable unless the seed is recorded	Every failure report must include the seed
Overkill for trivial blocks	For a 20-line module the testbench would be ten times the size of the design

1.4 Direct Versus Layered: Comparison

Aspect	Direct testbench	Layered testbench
Built from	One module, tasks, `initial` blocks	Classes inside a package, plus one small top module
Level of abstraction	Signal level throughout	Transaction level above the driver and monitor

Aspect	Direct testbench	Layered testbench
Stimulus	Hand-written vectors	Constrained-random transactions (plus directed tests when needed)
Expected values	Calculated by the engineer for each vector	Calculated by a reference model for every transaction
Checking	At the points the test chooses	Continuous, for every transaction the monitor sees
Access to DUT pins	Any code in the module	Only driver and monitor, through a virtual interface
Timing control	Scattered through the test	Concentrated in driver, monitor and clocking blocks
Adding a test	Copy and edit the vectors	Extend the test class and change constraints
Effect of a protocol change	Every test is edited	Driver and monitor are edited
Reuse across tests, levels and projects	Almost none	High
Coverage	Not measured	Functional coverage on the monitored transactions
Threads	One	One per component
Effort for the first test	Minutes to hours	Days
Effort for the hundredth test	The same as the first	Minutes
Scales to large designs	No	Yes
Typical use	Small blocks, bring-up, reproducing one bug	Block-level and system-level verification in industry

Interview trap: "So a layered testbench cannot run directed tests?" It can. A directed test in a layered testbench is a test that supplies fixed transactions instead of random ones (Section 6.4 does exactly this). The stimulus is directed, but it still flows through the same driver, monitor and scoreboard, so it is still checked automatically. "Layered" describes the structure, not the randomness.

1.5 Why Industry Uses the Layered Architecture

1. **The state space is too large for hand-written tests.** Even this toy ALU has more than half a million input combinations per cycle. A real block has sequences, modes and configurations on top of that. Only automatic generation plus automatic checking can cover it.
2. **Verification takes most of the project effort**, so anything reused saves a lot. Components are written once and reused by hundreds of tests, by the subsystem and chip-level testbenches, and by the next generation of the design.
3. **Specifications change during the project.** With separation of concerns, a change is made in one place.
4. **Teams are large.** A fixed architecture with agreed component roles lets several people work on one testbench and lets a new engineer find their way around any testbench in the company.
5. **Sign-off needs numbers.** Coverage collected at the monitor answers "what has been tested?" objectively.
6. **It is the basis of UVM.** UVM standardises exactly this structure (sequence, sequencer, driver, monitor, agent, scoreboard, environment, test). Understanding the plain SystemVerilog version makes UVM easy to learn, and interviewers for fresher roles test the plain version to check that understanding.

> **How to say it in an interview:** "A direct testbench drives hand-written vectors at pin level and checks them against hand-computed values, all in one module. It is fine for small blocks but does not scale: the tests are not reusable and only cover what the engineer thought of. A layered testbench splits the work into components: a generator creates random transactions, a driver converts them to pin activity through a virtual interface, a monitor converts pin activity back into transactions, and a scoreboard checks them against a reference model. The components communicate through mailboxes at transaction level, so stimulus, protocol timing and checking are independent of each other. That gives reuse, automatic checking of random stimulus, and small tests, which is why industry, and UVM, use this structure."

Interview questions

Q1. What is a layered testbench? A testbench organised as separate components at different levels of abstraction, typically generator, driver, monitor, scoreboard, environment and test, which exchange transaction objects through mailboxes. Only the driver and the monitor access DUT signals, through a virtual interface.

Q2. Name the layers. From the bottom: signal layer (interface, DUT), command layer (driver, monitor), functional layer (scoreboard, reference model), scenario layer (generator), and test layer.

Q3. Give three disadvantages of a direct testbench. Expected values are hand-computed for every vector; only scenarios the author thought of are tested; stimulus, timing and checking are mixed, so any protocol change breaks every test and nothing is reusable.

Q4. Give two disadvantages of a layered testbench. It costs much more to set up before the first test runs, and it is harder to debug because of multiple threads and because the testbench itself (especially the reference model) can contain bugs.

Q5. Which components know about the pin-level protocol? Only the driver and the monitor (and the interface they use). The generator, scoreboard and test work purely with transactions.

Q6. What is meant by "separation of concerns" in a testbench? Each component has one responsibility and hides how it does it. What to send (generator), how to send it (driver), what was observed (monitor) and whether it was right (scoreboard) can each be changed without touching the others.

Q7. When would you still write a direct testbench? For a very small block, for first bring-up of new RTL, or to reproduce one specific failing scenario quickly.

Q8. What is the difference between transaction level and signal level? At transaction level one object describes a complete operation (operands, opcode, result) with no notion of clock edges. At signal level the same operation is a set of individual signal values at specific clock edges. The driver and monitor convert between the two.

Q9. Is a layered testbench always constrained-random? No. The architecture is independent of the kind of stimulus. Directed transactions can be sent through the same components and are still checked automatically.

Q10. How does this architecture relate to UVM? UVM is a standard class library that implements the same architecture. The generator becomes a sequence and sequencer, driver and monitor keep their names and are grouped in an agent, the mailboxes become TLM ports, and the environment and test become `uvm_env` and `uvm_test`.

Part 2: The Project at a Glance and Its Foundation Files

The syllabus lists the testbench top first, because it is the outermost box. It is explained last in this module (Section 5.4), because it instantiates and starts everything else and can only be read once the other files are known. This part gives the overall picture and then the three files everything else depends on: the DUT, the interface and the transaction.

2.1 What Is Being Verified

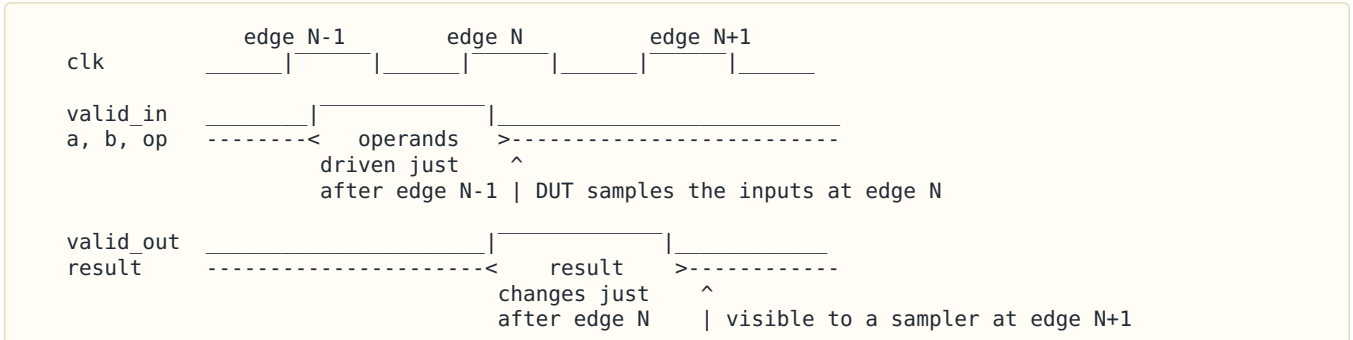
The DUT specification (from Module 1)

- Inputs `a` and `b` are 8 bits. Input `op` selects one of eight operations.
- When `valid_in` is 1 at a rising clock edge, the ALU computes the result and presents it **on the next cycle** with `valid_out` = 1.
- `result` is 9 bits: bit 8 is the carry of an addition, the borrow of a subtraction, or the bit shifted out by a left shift.
- `rst_n` is an asynchronous, active-low reset. During reset `valid_out` is 0 and `result` is 0.

`op`	Name	`result`
000	`ALU_ADD`	`a + b` (bit 8 is the carry)
001	`ALU_SUB`	`a - b` (bit 8 is the borrow)
010	`ALU_AND`	`a & b`
011	`ALU_OR`	`a | b`
100	`ALU_XOR`	`a ^ b`
101	`ALU_NOT`	`~a`
110	`ALU_SHL`	`a` shifted left by one (bit 8 receives the old `a[7]`)
111	`ALU_SHR`	`a` shifted right by one

The protocol in one picture

The testbench is built around this timing. Every later section refers back to it.



The rule that the monitor and scoreboard depend on: **inputs sampled at edge N belong with the output sampled at edge N+1.** The latency is exactly one clock cycle.

What the testbench must check

A small verification plan (Module 6 covers planning in general):

#	Feature	How it is checked in this project
1	Each of the eight operations gives the specified result	Scoreboard reference model `alu_predict`, for every transaction
2	Carry, borrow and shift-out appear in bit 8	Same reference model, computing in 9 bits; operands 0 and 255 are made more likely by a constraint
3	`valid_out` follows `valid_in` by exactly one cycle	Monitor: every sampled input must be followed by `valid_out` on the next edge, and `valid_out` must never appear without one
4	Outputs are never X when valid	Scoreboard compares with `===`; X fails the comparison
5	Reset clears `valid_out`	Monitor ignores activity during reset; a `valid_out` right after reset is reported as unexpected
6	All checked transactions are accounted for	End-of-test check: number checked equals number sent; a summary is printed

Functional coverage ("were all eight operations and the corner operands actually generated?") is added in Module 10; Exercise 3 of this module shows where it plugs in.

2.2 Who Creates Whom, and Who Shares What

Object hierarchy

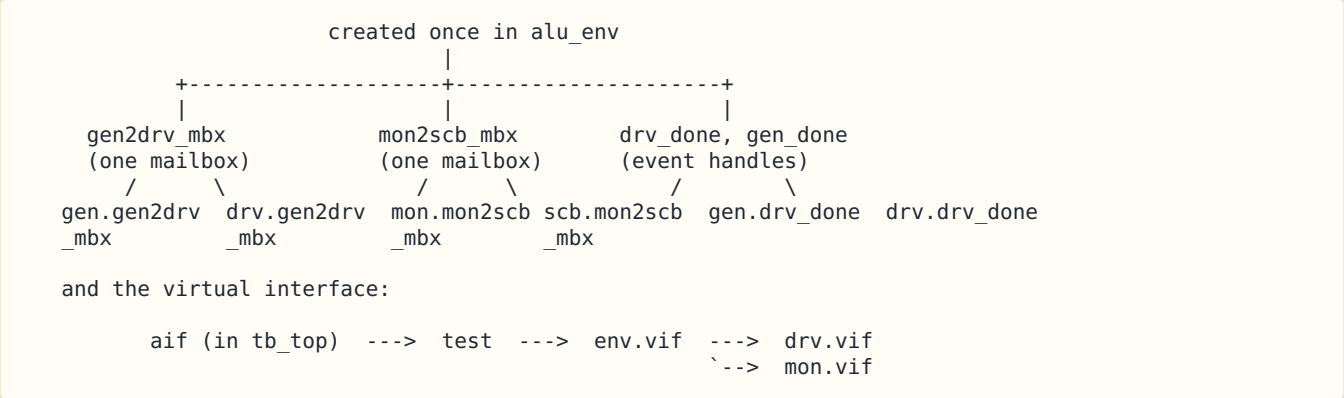
Static things (modules, interfaces) exist from the start of simulation. Class objects exist only after `new()` is called. The chain of construction is:

```
tb_top  (module, static)
  |-- aif            : alu_if instance           (static)
  |-- dut            : alu instance              (static)
  `-- initial block  : test = new(aif);          (dynamic from here down)
        |
        `-- alu_test.new(vif)
              `-- env = new(vif)                          alu_env
                    |-- gen2drv_mbx = new()                mailbox #(alu_txn)
                    |-- mon2scb_mbx = new()                mailbox #(alu_txn)
                    |-- gen = new(gen2drv_mbx, drv_done, gen_done)      alu_generator
                    |     `-- blueprint = new()                         alu_txn
                    |-- drv = new(vif, gen2drv_mbx, drv_done)           alu_driver
                    |-- mon = new(vif, mon2scb_mbx)                     alu_monitor
                    `-- scb = new(mon2scb_mbx)                          alu_scoreboard
```

Each object is created by the level above it, in its constructor. Nothing is created inside a `run()` task except transactions.

Shared handles

Two components can communicate only if they hold handles to the **same** object. The environment creates each shared object once and passes the handle to both sides.



> **Key point:** A mailbox variable, an event variable and a virtual interface variable are all *handles*. Copying the handle does not copy the mailbox, the event or the interface; it creates a second name for the same one. That is what makes them usable as communication channels.

Build, connect, run

Every class-based testbench goes through the same three stages, and it helps to know which code belongs to which:

Stage	What happens	Where in this project
Build	Objects are created with `new()`	Constructors of `alu_test` and `alu_env`
Connect	Shared handles (mailboxes, events, `vif`) are passed to the components	The arguments of the component constructors, in `alu_env::new`
Run	Each component's `run()` task executes as a parallel thread	`alu_env::run`, inside `fork` ... `join_any`

In this project build and connect happen together in the constructors. UVM separates them into `build_phase` and `connect_phase`.

2.3 The DUT Files

These two files are repeated from Module 1 without any change, so that this module's example folder is complete.

alu_pkg.sv

```
package alu_pkg;

  typedef enum logic [2:0] {
    ALU_ADD = 3'b000,
    ALU_SUB = 3'b001,
    ALU_AND = 3'b010,
    ALU_OR  = 3'b011,
    ALU_XOR = 3'b100,
    ALU_NOT = 3'b101,
    ALU_SHL = 3'b110,
    ALU_SHR = 3'b111
  } alu_op_e;

endpackage
```

alu.sv

```
module alu
  import alu_pkg::*;
(
  input  logic       clk,
  input  logic       rst_n,
  input  logic       valid_in,
  input  logic [7:0] a,
  input  logic [7:0] b,
  input  alu_op_e    op,
  output logic       valid_out,
  output logic [8:0] result
);

  logic [8:0] result_nxt;

  always_comb begin
    unique case (op)
      ALU_ADD: result_nxt = a + b;
      ALU_SUB: result_nxt = a - b;
      ALU_AND: result_nxt = {1'b0, a & b};
      ALU_OR : result_nxt = {1'b0, a | b};
      ALU_XOR: result_nxt = {1'b0, a ^ b};
      ALU_NOT: result_nxt = {1'b0, ~a};
      ALU_SHL: result_nxt = {a, 1'b0};
      ALU_SHR: result_nxt = {2'b00, a[7:1]};
    endcase
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      valid_out <= 1'b0;
      result    <= '0;
    end
    else begin
      valid_out <= valid_in;
      if (valid_in) result <= result_nxt;
    end
  end

endmodule
```

What matters for the testbench

Property of the DUT	Consequence for the testbench
Inputs are sampled at the rising edge when `valid_in` is 1	The driver must have the inputs stable before a rising edge and hold them through it
`valid_out <= valid_in;`	`valid_out` is `valid_in` delayed by one cycle; the monitor pairs input edge N with output edge N+1

Property of the DUT	Consequence for the testbench
`if (valid_in) result <= result_nxt;`	`result` keeps its old value in idle cycles; it must only be checked when `valid_out` is 1
The package holds `alu_op_e`	The testbench imports the same package, so DUT and testbench share one definition of the opcodes
The opcode type is shared with the DUT	The reference model must **not** copy the DUT's `case` statement; it is written independently from the specification (Section 4.2)

Module 1, Section 1.4 explains the design itself line by line.

2.4 The Interface: alu_if.sv

Why the testbench needs an interface

Module 8 covered this in depth; the short version:

1. **Bundling.** The DUT has eight ports. Without an interface, every one would be declared in the top module, connected to the DUT, and somehow made reachable from the classes.
2. **A class cannot have ports.** A class object is created at run time; it cannot be wired to signals the way a module instance is. The only way a class can reach static signals is through a **virtual interface**: a handle that points to an interface instance.
3. **Timing in one place.** Clocking blocks inside the interface define when signals are sampled and driven relative to the clock, so that the driver and monitor cannot race with the DUT.
4. **Access control.** Modports state which signals each user may read or write.

The file

alu_if.sv

```
interface alu_if (input logic clk);
  import alu_pkg::*;

  logic       rst_n;
  logic       valid_in;
  logic [7:0] a;
  logic [7:0] b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  // Driver's view: drive the DUT inputs, sample the DUT outputs
  clocking drv_cb @(posedge clk);
    default input #1step output #1;
    output valid_in, a, b, op;
    input  valid_out, result;
  endclocking

  // Monitor's view: sample everything, drive nothing
  clocking mon_cb @(posedge clk);
    default input #1step;
    input valid_in, a, b, op, valid_out, result;
  endclocking

  modport DUT (input clk, rst_n, valid_in, a, b, op, output valid_out, result);
  modport DRV (clocking drv_cb, input clk, rst_n);
  modport MON (clocking mon_cb, input clk, rst_n);

endinterface
```

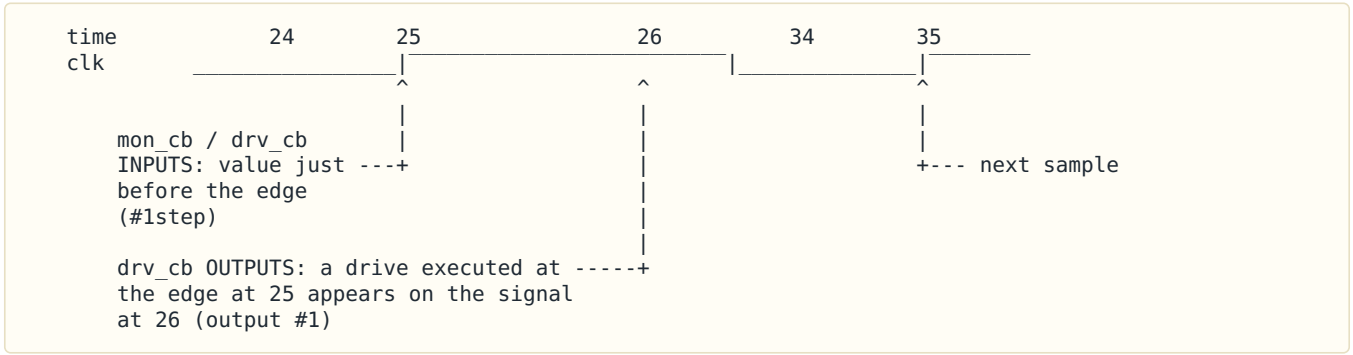
Line by line

Code	Explanation
`interface alu_if (input logic clk);`	The clock enters as a port, because it is generated outside (in `tb_top`) and shared with the DUT.
`import alu_pkg::*;`	Needed for the type `alu_op_e`.
`logic rst_n;` ... `logic [8:0] result;`	One signal for every DUT port except the clock. All are four-state so that X and Z from the DUT stay visible (Module 1, Section 2.1).
`alu_op_e op;`	The opcode signal has the enum type, so waveforms show names.
`clocking drv_cb @(posedge clk);`	A clocking block: a named group of signals with timing defined relative to the rising edge of `clk`.
`default input #1step output #1;`	Inputs are sampled one "step" before the edge, that is, the value the signal had just before the edge, before anything changes at the edge. Outputs are driven 1 time unit after the edge.
`output valid_in, a, b, op;`	Directions are from the testbench's point of view: the driver *drives* these (they are DUT inputs).
`input valid_out, result;`	The driver may sample the DUT outputs. Our driver does not need them, but a driver for a protocol with back-pressure (`ready`) would.
`clocking mon_cb ...` with only `input`	The monitor is passive. Giving it a clocking block with no outputs makes it impossible for monitor code to drive a signal by accident.
`modport DUT (...)`	The direction of each signal as seen by the design. The `alu` module of this course has ordinary ports, so this modport is not used by it; it is there for a DUT written with an interface port (`module alu (alu_if.DUT bus);`).

Code	Explanation
`modport DRV (clocking drv_cb, input clk, rst_n);`	The driver's view: the clocking block, plus read access to `clk` and `rst_n`.
`modport MON (clocking mon_cb, input clk, rst_n);`	The monitor's view.

`rst_n` is in no clocking block. Reset is asynchronous: it is driven directly by `tb_top` and read directly (`vif.rst_n`) by the driver and monitor.

What the clocking blocks do to timing



Action in class code	When it takes effect
`@(vif.drv_cb);` or `@(vif.mon_cb);`	The process resumes at the rising edge of `clk`, after the clocking block has taken its input samples
Reading `vif.mon_cb.valid_out`	Returns the value sampled at the most recent rising edge: the value the signal had immediately **before** that edge
`vif.drv_cb.valid_in <= 1'b1;` executed at an edge	The signal `valid_in` changes 1 time unit **after** that edge. The DUT sees the new value at the **following** rising edge

Two consequences that the driver and monitor rely on:

- The DUT's flip-flops and the monitor both sample at the same edge, and both see the values from before the edge. The monitor therefore sees exactly what the DUT sees. There is no race.
- A value the driver writes at edge N-1 is sampled by the DUT at edge N. The driver is always one edge ahead of the DUT.

> **Common mistake:** Reading the raw signal (`vif.valid_out`) in the monitor instead of the clocking block signal (`vif.mon_cb.valid_out`). At a clock edge the raw signal may or may not have been updated yet by the DUT's nonblocking assignment, depending on the order in which the simulator runs the two processes. The clocking block sample is always the pre-edge value.

> **Common mistake:** Driving through a clocking block with a blocking assignment: `vif.drv_cb.valid_in = 1;`. Clocking block outputs can only be driven with `<=` (a "synchronous drive"). The `=` form is a compile error.

Virtual interface

In every component, the handle is declared as

```
virtual alu_if vif;
```

and it receives its value through the constructor chain that starts in `tb_top` with `new(aif)`. Until it is assigned, `vif` is `null`, and any access such as `vif.drv_cb` is a fatal run-time error. Module 8 explains virtual interfaces in detail.

This project declares the handles as plain `virtual alu_if`, which gives a component access to everything in the interface. A stricter style declares `virtual alu_if.DRV vif;` in the driver and `virtual alu_if.MON vif;` in the monitor, so that the compiler enforces the modport. The plain form is used here because it keeps the constructor chain simple (one handle type from top to bottom) and is the most widely supported.

2.5 The Transaction: alu_txn.sv

Why a transaction class

A transaction is one complete operation of the DUT, described without any reference to clock edges: "ADD 200 and 100, result 300". It is the unit that travels through the mailboxes. Making it a class gives it three things a struct cannot have (Module 1, Section 3.3): randomization with constraints, methods, and extension by inheritance. Module 6 develops the transaction class step by step; here is its final form for this project.

What goes into a transaction

Kind of field	In `alu_txn`	Randomized?	Filled in by
Stimulus (DUT inputs)	`a`, `b`, `op`	Yes (`rand`)	Generator (driven side); monitor (observed side)
Response (DUT outputs)	`result`	No	Monitor
Bookkeeping	`id`	No	Generator or monitor
Control signals	None: `valid_in` and `valid_out` are protocol signals, not data. They are implied by the existence of the transaction		

Interview trap: "Do you put `clk`, `rst_n`, `valid_in` in the transaction?" No. Clock and reset are not part of an operation. `valid_in` is how the driver *expresses* a transaction on the pins; it is protocol, and protocol belongs to the driver and monitor.

The file

alu_txn.sv

```
class alu_txn;

  rand bit [7:0] a;
  rand bit [7:0] b;
  rand alu_op_e  op;
  logic [8:0]    result;
  int unsigned   id;

  // Make the corner operands 0 and 255 more likely than uniform random would
  constraint operand_c {
    a dist {8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8};
    b dist {8'h00 := 1, 8'hFF := 1, [8'h01:8'hFE] :/ 8};
  }

  function void print(string tag = "");
    $display("[%0t] %s id=%0d op=%s a=%0d b=%0d result=%0d",
             $time, tag, id, op.name(), a, b, result);
  endfunction

  virtual function alu_txn copy();
    alu_txn c;
    c        = new();
    c.a      = a;
    c.b      = b;
    c.op     = op;
    c.result = result;
    c.id     = id;
    return c;
  endfunction

  function bit compare(alu_txn rhs);
    if (rhs == null) return 0;
    return (a == rhs.a) && (b == rhs.b) && (op == rhs.op) && (result === rhs.result);
  endfunction

endclass
```

There is no `package` or `import` in this file. It is not compiled on its own: it is pulled into the package `alu_tb_pkg` with `` `include `` (Section 5.3), and the package imports `alu_pkg`. The same is true of every class file in this project.

Line by line

Code	Explanation
`rand bit [7:0] a;`	Two-state, because random values are never X or Z. `rand` means `randomize()` chooses a new value each time.
`rand alu_op_e op;`	A random enum only ever receives one of its eight named values, so every generated opcode is legal.
`logic [8:0] result;`	Not `rand`: it is an output of the DUT. Four-state, so that an X from the DUT is preserved all the way to the scoreboard. Its default value is X, which correctly means "not known yet".
`int unsigned id;`	A serial number for matching log lines. Not random.
`constraint operand_c {...}`	A distribution: weight 1 for `00`, weight 1 for `FF`, and a total weight of 8 shared by the 254 values between them. So each operand is 0 about 10 percent of the time and 255 about 10 percent of the time. Carry, borrow and shift-out bugs live at those values. Module 5 covers `dist`.

Code	Explanation
`function void print(string tag = "");`	Prints all fields on one line. The `tag` says which component is printing (`GEN`, `DRV`, `MON`). Having one print routine keeps the log format identical everywhere.
`op.name()`	The enum method that returns the name as a string.
`result=%0d`	Prints `x` when the result has not been filled in yet, which is the case for transactions on the stimulus side.
`virtual function alu_txn copy();`	Returns a new object with the same field values (a deep copy; this class contains no handles, so copying the fields is enough). `virtual` lets a derived transaction with extra fields override it.
`alu_txn c; c = new();`	Declaration and construction are separate statements.
`function bit compare(alu_txn rhs);`	Returns 1 if the two transactions match.
`if (rhs == null) return 0;`	Protects against a null handle.
`result === rhs.result`	Case equality: an X in either result gives a definite "not equal" instead of an unknown. The inputs are two-state (`a`, `b`) or generated (`op`), so `==` is sufficient for them.
`id` is not compared	It is bookkeeping, not DUT behaviour.

How the same class is used on both sides

```
   Stimulus side                          Observation side

   generator:                             monitor:
     blueprint.randomize()                  t = new()
     txn = blueprint.copy()                 t.a, t.b, t.op <- sampled pins
     a, b, op = random values               t.result       <- sampled pins
     result   = x  (unknown)                (a complete record of what happened)
          |                                      |
          v                                      v
   driver:                                scoreboard:
     uses a, b, op                          expected = t.copy()
     ignores result                         expected.result = alu_predict(t)
                                            t.compare(expected)
```

One class serves both paths. On the stimulus side only the input fields are meaningful. On the observation side all fields are filled from the pins.

> **Common mistake:** Declaring `result` as `rand`. The solver would then invent a result, and a careless scoreboard might compare the DUT against a random number, or worse, against itself. Only fields the testbench is free to choose are `rand`.

> **Common mistake:** Declaring `result` as `bit [8:0]`. An X on the DUT output would silently become 0 in the monitor, and for an operation whose correct answer is 0 the scoreboard would report PASS.

Interview questions

Q1. What is a transaction? An object that describes one complete operation on the DUT at a level above the pins: the stimulus fields, the response fields and any bookkeeping. It is what the testbench components pass to each other.

Q2. Which fields of a transaction are declared `rand`? Only the fields that the testbench is free to choose, that is, the DUT inputs (`a`, `b`, `op`). Outputs of the DUT (`result`) and bookkeeping fields (`id`) are not random.

Q3. Why is `result` a `logic` and `a` a `bit`? `a` is stimulus: it is generated, never X, and two-state is faster and randomizes cleanly. `result` is sampled from the DUT and must be able to hold X or Z so that the scoreboard can detect them.

Q4. Why does the transaction have a `copy()` method? Because assigning one handle to another does not copy the object. The generator needs a fresh, independent object for each transaction it sends, and the scoreboard needs an independent object to hold the expected values.

Q5. Why is `===` used inside `compare()`? So that an X or Z in the observed result makes the comparison fail with a definite 0. With `==` the comparison would evaluate to X, which could be misread as a pass or silently skipped.

Q6. Why are the class files not compiled directly? They are textually included into a package. A class that is declared inside a package can be imported wherever it is needed, and every user sees the same type. Compiling the same class file in two places would create two different, incompatible types.

Q7. What do the two clocking blocks in the interface give you? A race-free timing discipline. Inputs are sampled just before the clock edge (`#1step`), so the testbench sees the same values the DUT sees. Outputs are driven a fixed skew after the edge, so the DUT never sees a testbench drive at the same instant as its own sampling. Having a separate input-only block for the monitor prevents the monitor from driving anything.

Q8. Why can a class not connect to an interface instance directly? A class is a dynamic object created at run time and has no ports, while an interface instance is a static part of the design hierarchy. The class holds a `virtual interface` handle that is assigned, at run time, to point to the instance.

Q9. Who drives `rst_n`? The top module. Reset is a property of the environment around the DUT, not of an individual transaction, and it must be active before any class object starts running.

Part 3: The Stimulus Side: Generator and Driver

3.1 The Generator

In plain English

The generator is the component that decides **what** to send to the DUT. It produces a stream of transaction objects, usually randomized, and hands them one at a time to the driver. It knows nothing about clocks, pins or timing. If you deleted the DUT and the interface, the generator would still compile.

Technical definition

A generator is a class that creates transaction objects, randomizes them (subject to the constraints in the transaction class and any the test adds), and passes them to the driver through a mailbox. It belongs to the scenario layer. In UVM the same role is played by a *sequence*.

Why it is a separate component

If stimulus generation were inside the driver...	With a separate generator
A new kind of stimulus would mean editing the driver	A test changes or replaces the generator; the driver is untouched
The driver could not be reused with a different stimulus source	The driver drives whatever arrives in its mailbox
Random and directed stimulus would need two drivers	Two generators (or two tests), one driver

Why a mailbox between generator and driver

The generator works in zero simulation time: it can create a thousand transactions without a single clock edge passing. The driver needs two clock cycles per transaction. Two processes running at different speeds need a channel that

1. carries an object from one to the other,
2. makes the receiver wait when nothing is available, and
3. keeps the two sides from knowing about each other.

A mailbox does all three (Module 7). `put()` places a handle in the mailbox. `get()` removes one, and blocks the caller while the mailbox is empty. The generator never calls a driver method and the driver never calls a generator method; each only knows the mailbox.

Keeping the generator in step with the driver

With an unbounded mailbox, `put()` never blocks, so a generator that just loops would create all its transactions at time 0. That works, but it has two drawbacks: all objects exist in memory at once, and the generator cannot react to anything that happens during the test. Three common ways to pace the generator:

Method	How	Used here?
Handshake event	Generator waits for an event (`drv_done`) that the driver triggers after each transaction	Yes
Bounded mailbox	`new(1)`: `put()` blocks while the mailbox is full, so the generator is at most one transaction ahead	Alternative
No pacing	Generate everything at time 0 into an unbounded mailbox	Simplest, common in small examples

The blueprint pattern

The obvious generator loop constructs a new transaction each time:

```
repeat (num_txns) begin
  txn = new();                    // the class of the object is fixed here: alu_txn
  if (!txn.randomize()) ...
  gen2drv_mbx.put(txn);
end
```

The weakness is the line `txn = new();`. It hard-codes the class `alu_txn` inside the generator. A test that wants a transaction with extra constraints would have to edit the generator.

The **blueprint pattern** removes that. The generator owns one object, the blueprint. It randomizes the *blueprint* each time and sends a *copy*. A test can replace the blueprint with an object of a derived class before the run starts, and from then on every randomization obeys the derived class's constraints, with no change to the generator.

```
test (optional):  gen.blueprint = <object of a class derived from alu_txn>

generator loop:   blueprint.randomize()    <- constraints of the object's ACTUAL class
                  txn = blueprint.copy()   <- a fresh object for this transaction
                  put(txn)
```

Section 6.6 uses this to build `alu_arith_test`.

The file

alu_generator.sv

```
class alu_generator;

  mailbox #(alu_txn) gen2drv_mbx;
  event              drv_done;
  event              gen_done;
  alu_txn            blueprint;
  int unsigned       num_txns = 10;
  int unsigned       num_sent;

  function new(mailbox #(alu_txn) gen2drv_mbx, event drv_done, event gen_done);
    this.gen2drv_mbx = gen2drv_mbx;
    this.drv_done    = drv_done;
    this.gen_done    = gen_done;
    blueprint        = new();
  endfunction

  // Send one transaction to the driver and wait until the driver has applied it
  task send(alu_txn txn);
    num_sent++;
    txn.id = num_sent;
    gen2drv_mbx.put(txn);
    txn.print("GEN");
    @(drv_done);
  endtask

  virtual task run();
    repeat (num_txns) begin
      if (!blueprint.randomize()) $fatal(1, "GEN: randomization failed");
      send(blueprint.copy());
    end
    -> gen_done;
  endtask

endclass
```

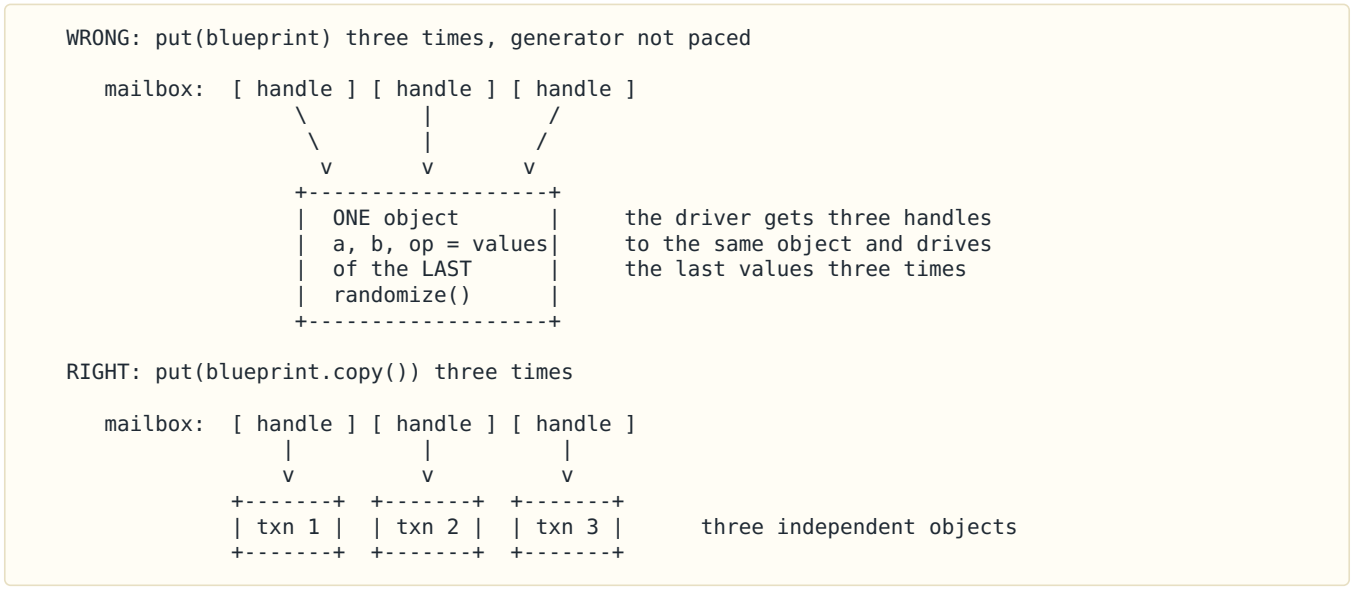
Line by line

Code	Explanation
`mailbox #(alu_txn) gen2drv_mbx;`	A handle to a parameterized mailbox that can only carry `alu_txn` objects (or objects of classes derived from it). The compiler rejects an attempt to put anything else in. The generator does **not** create the mailbox; it receives the handle.
`event drv_done;`	Handle to the event the driver triggers when it has finished applying a transaction.
`event gen_done;`	Handle to an event the generator triggers once, after the last transaction.
`alu_txn blueprint;`	The object that is randomized. Public, so that a test can replace it.
`int unsigned num_txns = 10;`	How many transactions to generate. A default is given; the test overrides it.
`int unsigned num_sent;`	How many have actually been sent. The environment uses it at the end of the test.
`function new(mailbox #(alu_txn) gen2drv_mbx, event drv_done, event gen_done);`	The constructor receives the shared handles. This is the "connect" step.
`this.gen2drv_mbx = gen2drv_mbx;`	The argument has the same name as the class property; `this.` selects the property (Module 4). After this line the generator and whoever else holds this handle share one mailbox.

Code	Explanation
`this.drv_done = drv_done;`	Assigning an event to an event makes both names refer to the same event.
`blueprint = new();`	A default blueprint of the base class, so the generator works even if the test supplies nothing.
`task send(alu_txn txn);`	A task, because it waits. Both `run()` here and the directed generator in Section 6.4 call it.
`num_sent++; txn.id = num_sent;`	Numbers the transactions 1, 2, 3...
`gen2drv_mbx.put(txn);`	Places the handle in the mailbox. The mailbox is unbounded, so this never blocks.
`txn.print("GEN");`	Log line.
`@(drv_done);`	Blocks until the driver triggers the event. The generator is now idle until the transaction has been applied to the DUT.
`virtual task run();`	`virtual`, so that a derived generator can replace the stimulus loop (Section 6.4).
`if (!blueprint.randomize()) $fatal(...)`	`randomize()` returns 0 if the constraints cannot be satisfied. The return value must always be checked; an unchecked failure leaves the old values in the object and the test carries on with stale data. `$fatal` ends the simulation because nothing useful can follow.
`send(blueprint.copy());`	A copy is sent, never the blueprint itself.
`-> gen_done;`	Announces that all stimulus has been generated. Then `run()` returns, which is what ends the `fork` ... `join_any` in the environment (Section 5.1).

Why a copy is sent and not the blueprint

A mailbox stores **handles**, not objects. If the generator put the blueprint itself into the mailbox, every entry would be a handle to the same single object, and the next `randomize()` would overwrite the values of the transaction the driver is still using.



With the `drv_done` handshake in this generator the overwrite would not actually happen (the generator waits before randomizing again), but the driver, the log and any later consumer would still all be holding the one blueprint object. Sending a copy is the habit that is always safe.

Is there a race on drv_done?

An event trigger `->` is only seen by processes that are already waiting. If the driver triggered `drv_done` before the generator reached `@(drv_done)`, the generator would wait forever. Here that cannot happen: after `put()` the generator reaches `@(drv_done)` without any time passing, and the driver needs two clock edges after its `get()` before it triggers the event. The trigger always comes at least one clock edge after the generator has started to wait.

> **Common mistake:** Forgetting to check the return value of `randomize()`. With conflicting constraints the call fails, returns 0, and leaves the variables unchanged. The test then appears to run normally, sending the same transaction over and over.

> **Common mistake:** Calling `new()` on the mailbox inside the generator. The generator would then own a private mailbox that nobody reads, and the driver would wait forever on a different one. Shared objects are created once, in the environment.

> **Verification application:** In a real environment the generator is the most frequently varied component. There is typically a library of generators (in UVM, sequences): one random, one for error injection, one that replays a recorded trace, one per directed scenario. All of them feed the same driver.

Interview questions

Q1. What is a generator? The component that creates transaction objects, randomizes them, and sends them to the driver through a mailbox. It works at transaction level and has no access to the interface.

Q2. Why does the generator not drive the DUT directly? Because deciding what to send and knowing how to put it on the pins are separate concerns. Keeping them apart lets the stimulus be changed without touching the driver, and lets the driver be reused with any stimulus source.

Q3. Why do you create a new object (or a copy) for every transaction? A mailbox holds handles. If the same object were put repeatedly, all the handles would point to one object whose contents keep changing with every `randomize()`, so earlier transactions would be overwritten before the driver used them.

Q4. What is the blueprint pattern and what problem does it solve? The generator randomizes one template object and sends copies of it. Because the template is a handle that a test may replace with an object of a derived class, a test can change the constraints (or add fields) without modifying the generator code. It removes the hard-coded `new()` of a specific class from the generator.

Q5. How does the generator know when to produce the next transaction? In this testbench it waits on the event `drv_done`, which the driver triggers after each transaction. Alternatives are a bounded mailbox of size 1, whose `put()` blocks until the driver has taken the previous item, or no pacing at all.

Q6. What happens if `randomize()` fails? It returns 0 and the random variables keep their previous values. The generator must check the return value and stop the test with a clear message.

Q7. Does the generator consume simulation time? Not by itself. Creating and randomizing objects takes zero simulation time. It only waits because of the handshake with the driver.

Q8. How does the generator signal that it has finished? Its `run()` task returns, which ends the `fork`...`join_any` in the environment, and it triggers the event `gen_done` for any component that wants to wait for that explicitly.

3.2 The Driver

In plain English

The driver is the component that knows **how** to talk to the DUT. It takes one transaction at a time from the mailbox and turns it into the right signal values at the right clock edges. It is the only component that writes to the DUT's input pins.

Technical definition

A driver is a class that receives transactions from the generator through a mailbox and converts each one into pin-level activity on the DUT inputs, following the DUT's protocol and synchronised to the clock, through a virtual interface. It belongs to the command layer. In UVM it is a `uvm_driver`; a driver with this role is also called a bus functional model (BFM).

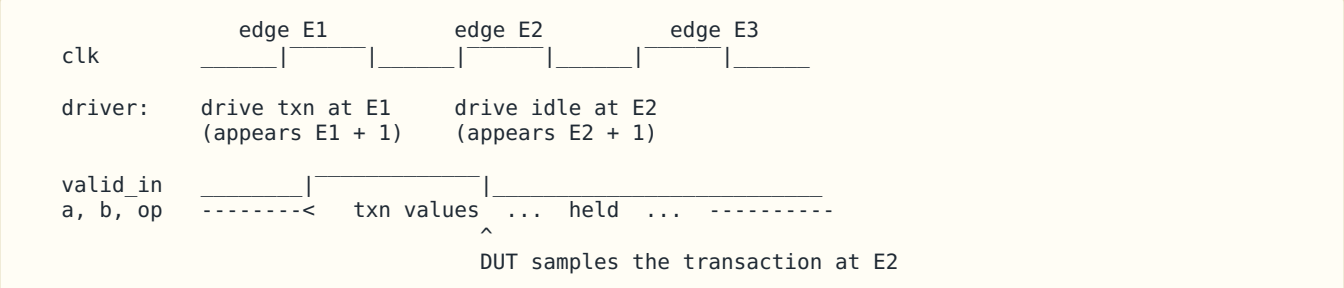
What the driver does and does not do

The driver does	The driver does not
Get the next transaction from the mailbox	Create or randomize transactions
Wait for the correct clock edge	Decide what data to send
Drive `valid_in`, `a`, `b`, `op` for the right number of cycles	Check DUT outputs
Return the bus to idle afterwards	Predict results
Drive idle values during and after reset	Generate the clock or the reset
Tell the generator when it is done (`drv_done`)	Send anything to the scoreboard

> **Interview trap:** "The driver already knows what it drove. Why not have the driver send the transaction to the scoreboard, and save writing a monitor?" Because then the testbench checks what it *intended* to drive, not what actually reached the DUT. A bug in the driver, in the interface connections or in the protocol timing would be invisible. Section 4.1 returns to this.

The protocol the driver implements

For one transaction the ALU needs `valid_in` high, with stable operands, across exactly one rising clock edge:



The driver therefore needs two clocking events per transaction: one to start driving, and one at which the DUT samples and the driver removes `valid_in`.

How the driver reaches the pins

Recall from Section 2.4:

- `@(vif.drv_cb);` waits for the next rising edge of the clock.
- `vif.drv_cb.valid_in <= 1'b1;` is a synchronous drive. Executed at an edge, it changes the signal one time unit after that edge.

All driving goes through the clocking block. The driver never assigns to `vif.valid_in` directly.

The file

alu_driver.sv

```
class alu_driver;

  virtual alu_if     vif;
  mailbox #(alu_txn) gen2drv_mbx;
  event              drv_done;
  int unsigned       num_driven;

  function new(virtual alu_if vif, mailbox #(alu_txn) gen2drv_mbx, event drv_done);
    this.vif         = vif;
    this.gen2drv_mbx = gen2drv_mbx;
    this.drv_done    = drv_done;
  endfunction

  // Put the bus into its idle state, then wait until reset has been released
  task reset();
    @(vif.drv_cb);
    vif.drv_cb.valid_in <= 1'b0;
    vif.drv_cb.a        <= '0;
    vif.drv_cb.b        <= '0;
    vif.drv_cb.op       <= ALU_ADD;
    wait (vif.rst_n === 1'b1);
  endtask

  // Apply one transaction: valid_in is high across exactly one rising edge
  task drive(alu_txn txn);
    @(vif.drv_cb);
    vif.drv_cb.valid_in <= 1'b1;
    vif.drv_cb.a        <= txn.a;
    vif.drv_cb.b        <= txn.b;
    vif.drv_cb.op       <= txn.op;
    @(vif.drv_cb);                    // the DUT samples the transaction at this edge
    vif.drv_cb.valid_in <= 1'b0;
  endtask

  task run();
    alu_txn txn;
    reset();
    forever begin
      gen2drv_mbx.get(txn);
      drive(txn);
      num_driven++;
      txn.print("DRV");
      -> drv_done;
    end
  endtask

endclass
```

Line by line

Code	Explanation
`virtual alu_if vif;`	Handle to the interface instance. Null until the constructor assigns it.
`mailbox #(alu_txn) gen2drv_mbx;`	The same mailbox the generator writes to; the handle arrives through the constructor.
`event drv_done;`	The same event the generator waits on.
`int unsigned num_driven;`	A counter, useful when debugging a hung test ("how far did the driver get?").
`function new(virtual alu_if vif, ...)`	The constructor stores the three shared handles. Nothing else: no driving, no waiting. A constructor is a function and cannot consume time.
`task reset();`	Called once at the start of `run()`.
`@(vif.drv_cb);` (in `reset`)	Waits for the first rising clock edge. Clocking block drives are tied to clocking events, so the driver aligns itself to an edge before its first drive.
`vif.drv_cb.valid_in <= 1'b0;` and the next three lines	Idle values: nothing valid, operands zero, a legal opcode. They appear on the signals one time unit after the edge. From then on the DUT inputs are never X.
`wait (vif.rst_n === 1'b1);`	Level-sensitive wait until reset is released. If reset is already released, it does not block. `===` is used so that an X on `rst_n` does not count as released.
`task drive(alu_txn txn);`	Converts one transaction into pin activity. This task **is** the protocol. If the protocol changes, this is the code that changes.
`@(vif.drv_cb);` (first in `drive`)	Aligns to a clock edge. `get()` in `run()` may have returned at any time, so the driver must synchronise before driving.
`vif.drv_cb.valid_in <= 1'b1;` and three more	The transaction's fields are driven onto the pins. They change one time unit after this edge.
`@(vif.drv_cb);` (second)	One clock later. At this edge the DUT's flip-flops sample `valid_in` = 1 and the operands.
`vif.drv_cb.valid_in <= 1'b0;`	Removes `valid_in` one time unit after the sampling edge. The operands are left as they are; the DUT ignores them when `valid_in` is 0.
`alu_txn txn;` (in `run`)	A local handle. No `new()`: the driver does not create transactions, it receives them.
`reset();`	Idle the bus and wait for reset before taking any transaction.
`forever begin`	The driver never finishes by itself. The environment ends it (Section 5.1).
`gen2drv_mbx.get(txn);`	Blocks until a transaction is available, then removes it from the mailbox and stores its handle in `txn`.
`drive(txn);`	Takes two clock edges.
`txn.print("DRV");`	Printed at the edge where the DUT sampled the transaction.
`-> drv_done;`	Wakes the generator, which then produces the next transaction.

Step-by-step execution for the first transaction

With rising clock edges at 5, 15, 25... and reset released at time 20 (Section 5.4):

Time	Driver	Signals
0	`run()` starts, enters `reset()`, waits for a clock edge	`valid_in`, `a`, `b`, `op` are X (never driven yet); DUT is held in reset
5	Edge. Schedules the idle values	
6		`valid_in` = 0, `a` = 0, `b` = 0, `op` = `ALU_ADD`
6 to 20	Blocked in `wait (vif.rst_n === 1'b1)`	
20	Reset released; `reset()` returns. `get()` returns at once: the generator put transaction 1 in the mailbox at time 0. `drive()` starts and waits for an edge	
25	Edge. Schedules `valid_in` = 1 and the operands	
26		`valid_in` = 1, operands of transaction 1
35	Edge: the DUT samples the transaction. Driver schedules `valid_in` = 0, prints, triggers `drv_done`, loops to `get()`	
35	(same time step) Generator wakes, sends transaction 2. Driver's `get()` returns; `drive()` waits for an edge	
36		`valid_in` = 0
45	Edge. Schedules transaction 2	
46		`valid_in` = 1, operands of transaction 2
55	Edge: the DUT samples transaction 2	

So one transaction is applied every two clock cycles: one cycle with `valid_in` high, one idle cycle.

Why the first statement of drive() is a clock wait

Suppose `drive()` drove immediately and only then waited for an edge:

```
// Fragile version
vif.drv_cb.valid_in <= 1'b1;      // (1) executed whenever get() happened to return
...
@(vif.drv_cb);
vif.drv_cb.valid_in <= 1'b0;      // (2)
```

If `get()` returns between two clock edges, drive (1) is not at a clocking event. The standard says such a drive is processed as if it had been executed at the next clocking event. Drive (2) is executed exactly at that next event. Both drives then apply to `valid_in` at the same time, the later one wins, and `valid_in` never goes high: the transaction is silently lost. Waiting for the clocking event first guarantees that every drive is executed at an edge, and that two drives to the same signal are always at different edges.

Differences between generator and driver

Aspect	Generator	Driver
Layer	Scenario	Command

Aspect	Generator	Driver
Works with	Transaction objects only	Transactions in, signals out
Knows the pin protocol	No	Yes; it implements it
Has a virtual interface	No	Yes
Uses the clock	No	Yes, through the clocking block
Creates transactions	Yes	No
Mailbox operation	`put()`	`get()`
Finishes by itself	Yes, after `num_txns`	No, `forever` loop
Changed when	A new scenario is needed	The protocol changes
UVM equivalent	Sequence	Driver

Common mistake: Driving DUT inputs with plain blocking assignments at the active clock edge, for example `@(posedge vif.clk); vif.valid_in = 1;`. The DUT's `always_ff` is triggered by the same edge. Whether the DUT sees the old or the new value depends on which process the simulator runs first: a race. The result can differ between simulators, or change when an unrelated line is added. Driving through a clocking block (or, in simple benches, with a nonblocking assignment or on the opposite edge) removes the race.

Common mistake: Using `peek()` instead of `get()`. `peek()` copies the handle but leaves the transaction in the mailbox, so the driver would drive the first transaction forever.

Common mistake: Leaving the DUT inputs undriven (X) after reset because the driver only starts driving when the first transaction arrives. The DUT may then sample X on `valid_in`. A driver should drive idle values first.

Verification application: A driver for a protocol with back-pressure adds one more step: after asserting `valid`, it keeps the signals stable until it samples `ready` high through the clocking block (`while (!vif.drv_cb.ready) @(vif.drv_cb);`). That is the reason `drv_cb` also contains inputs.

Interview questions

Q1. What is a driver? The component that takes transactions from the generator through a mailbox and drives the DUT's input pins according to the protocol, synchronised to the clock, through a virtual interface.

Q2. How does the driver access the DUT signals? Through a virtual interface handle that is passed down from the top module at construction. It drives through the interface's clocking block with nonblocking (synchronous) drives.

Q3. Why does the driver use a clocking block? To avoid races with the DUT. The clocking block drives outputs a fixed skew after the clock edge and samples inputs just before it, so the DUT never samples a signal at the same instant the driver changes it.

Q4. What does the driver do when the mailbox is empty? `get()` blocks, so the driver simply waits. The bus stays in the idle state it was left in (`valid_in` = 0).

Q5. What is the difference between `get()` and `peek()` in the driver? `get()` removes the transaction from the mailbox. `peek()` returns a handle to it but leaves it there. A driver uses `get()` so that each transaction is driven once. (Some drivers `peek()`, drive, then `get()`, so that the item stays in the mailbox while it is in progress; with a bounded mailbox of size 1, that holds back the generator without needing an event.)

Q6. How does the driver tell the generator that a transaction is finished? It triggers the shared event `drv_done` after the DUT has sampled the transaction.

Q7. Does the driver create transaction objects? No. It declares a handle and receives objects through the mailbox.

Q8. What should the driver do during reset? Drive the inputs to their idle values and not start any transaction until reset is released.

Q9. Why is the driver's `run()` a `forever` loop? How does it stop? The driver does not know how many transactions the test will send; that is the test's decision. It keeps serving the mailbox until the environment ends the simulation phase, here by `disable fork` after the scoreboard has checked everything.

Q10. If the protocol changed so that `valid_in` had to be held for two cycles, which files would change? Only `alu_driver.sv` (the `drive()` task) and, because the pairing of inputs and outputs is protocol knowledge too, `alu_monitor.sv`. The generator, the scoreboard, the environment and every test stay as they are.

Part 4: The Checking Side: Monitor and Scoreboard

4.1 The Monitor

In plain English

The monitor is the testbench's eyes. It watches the DUT pins on every clock edge, recognises when an operation has taken place, and writes down what it saw as a transaction object: "inputs were 200, 100, ADD, and the result that came out was 300". It then sends that record to the scoreboard. It never drives anything.

Technical definition

A monitor is a passive class that samples the DUT's interface signals at the correct clock edge through a virtual interface, reassembles pin-level activity into transaction objects according to the protocol, and sends each observed transaction to the scoreboard (and to coverage collectors) through a mailbox. It belongs to the command layer.

Why a monitor is needed

The scoreboard works with transactions, but the DUT produces signal changes. Something has to translate back. More importantly, the checking path must be based on **what actually happened at the pins**, independent of what the stimulus side intended.

If the scoreboard used the driver's or generator's transaction as "what the DUT received"	With an independent monitor
A driver that drives the wrong operand, or drives one cycle late, goes unnoticed: the scoreboard predicts from the intended values and compares with a result computed from different values, and reports a DUT bug that is not there, or misses one that is	The prediction is made from the values the DUT really sampled
A mis-wired port in `tb_top` is blamed on the DUT	The monitor sees the same wires the DUT sees
The checker cannot be reused when the ALU sits inside a larger design, because there is no driver there	Monitor and scoreboard are reused unchanged: they only need the signals
Protocol errors on the pins (`valid_out` at the wrong time) are never looked at	The monitor checks the protocol on every edge

Driver versus monitor

Aspect	Driver	Monitor
Direction of conversion	Transaction → signals	Signals → transaction
Active or passive	Active: drives DUT inputs	Passive: only samples
Which signals	DUT inputs (may sample some outputs for handshaking)	Inputs and outputs

Aspect	Driver	Monitor
Clocking block	`drv_cb` (outputs and inputs)	`mon_cb` (inputs only)
Mailbox operation	`get()` from `gen2drv_mbx`	`put()` to `mon2scb_mbx`
Creates transaction objects	No	Yes, one new object per observed operation
Talks to	Generator (mailbox, `drv_done`)	Scoreboard (mailbox)
Activity when the bus is idle	Blocked on the mailbox	Still sampling every edge
Present when the block is reused inside a bigger design	No (real logic drives the block)	Yes
UVM: present in a passive agent	No	Yes

What the monitor has to get right

1. **Sample at the right moment.** It must see the same values the DUT's flip-flops see. Sampling through `mon_cb` (input skew `#1step`) gives the values from just before the clock edge, which is exactly that.
2. **Pair the right input with the right output.** The ALU has a one-cycle latency. The inputs sampled at edge N belong with the result sampled at edge N+1.
3. **Create a new object for each transaction.** Explained below.
4. **Ignore the bus during reset.**
5. **Check the protocol** while it is watching anyway.

The latency alignment

```
edge              N-1          N          N+1         N+2
                   |           |           |           |
sampled valid_in   0           1           0           1        (values seen by mon_cb
sampled a, b, op   -         txn1        (old)       txn2       at each edge)
sampled valid_out  0           0           1           0
sampled result     -           -         res1          -

monitor action:              create      complete    create
                             pending     pending     pending
                             = txn1      with res1,  = txn2
                             inputs      send it
```

The monitor keeps one handle, `pending`, for "an operation whose inputs I have seen and whose result I am waiting for". On each edge it does two things, in this order:

1. If there is a pending transaction, this edge must show `valid_out`. Take `result`, send the completed transaction, clear `pending`.
2. If `valid_in` was sampled high at this edge, create a new pending transaction from `a`, `b`, `op`.

The order matters when transactions arrive on consecutive cycles: the edge that carries the result of transaction 1 is also the edge that carries the inputs of transaction 2. Step 1 must finish with the old `pending` before step 2 replaces it.

The file

alu_monitor.sv

```
class alu_monitor;

  virtual alu_if     vif;
  mailbox #(alu_txn) mon2scb_mbx;
  int unsigned       num_seen;

  function new(virtual alu_if vif, mailbox #(alu_txn) mon2scb_mbx);
    this.vif         = vif;
    this.mon2scb_mbx = mon2scb_mbx;
  endfunction

  task run();
    alu_txn pending;      // inputs sampled at the previous edge, result still to come
    forever begin
      @(vif.mon_cb);

      if (vif.rst_n !== 1'b1) begin
        pending = null;   // nothing observed during reset counts
        continue;
      end

      // 1. Output side: complete the transaction whose inputs were seen one edge ago
      if (pending != null) begin
        if (vif.mon_cb.valid_out === 1'b1) begin
          pending.result = vif.mon_cb.result;
          pending.print("MON");
          mon2scb_mbx.put(pending);
        end
        else
          $error("[%0t] MON: valid_out missing for id=%0d", $time, pending.id);
        pending = null;
      end
      else if (vif.mon_cb.valid_out === 1'b1)
        $error("[%0t] MON: valid_out without a preceding valid_in", $time);

      // 2. Input side: start a new transaction if the DUT sampled one at this edge
      if (vif.mon_cb.valid_in === 1'b1) begin
        if ($isunknown({vif.mon_cb.a, vif.mon_cb.b, vif.mon_cb.op}))
          $error("[%0t] MON: X or Z on the DUT inputs while valid_in is 1", $time);
        pending    = new();
        num_seen++;
        pending.id = num_seen;
        pending.a  = vif.mon_cb.a;
        pending.b  = vif.mon_cb.b;
        pending.op = vif.mon_cb.op;
      end
    end
  endtask

endclass
```

Line by line

Code	Explanation
`virtual alu_if vif;`	Handle to the same interface instance the driver uses.
`mailbox #(alu_txn) mon2scb_mbx;`	The channel to the scoreboard.
`int unsigned num_seen;`	Counts observed operations; also used as the `id` of observed transactions. Because transactions are observed in the order they were sent, these ids match the generator's.

Code	Explanation
`alu_txn pending;`	A local handle, initially `null`. Class methods have automatic lifetime, so this variable belongs to this call of `run()`.
`forever begin @(vif.mon_cb);`	One iteration per rising clock edge, for the whole simulation. When the process resumes, the clocking block has already taken its samples.
`if (vif.rst_n !== 1'b1)`	Reset is read directly from the interface (it is asynchronous and not in a clocking block). `!==` makes an X on reset count as "in reset".
`pending = null; continue;`	Discard any half-observed operation and skip the rest of this iteration.
`if (pending != null)`	An input was sampled at the previous edge, so this edge must carry its result.
`vif.mon_cb.valid_out === 1'b1`	The sampled (pre-edge) value of `valid_out`. `===` so that X does not count as valid.
`pending.result = vif.mon_cb.result;`	Completes the record. `result` is `logic [8:0]` on both sides, so X bits are kept.
`mon2scb_mbx.put(pending);`	Sends the handle to the scoreboard.
`$error("... valid_out missing ...")`	Protocol check: the DUT did not answer after one cycle.
`pending = null;`	The monitor lets go of the object. The mailbox (and soon the scoreboard) still holds a handle, so the object stays alive.
`else if (vif.mon_cb.valid_out === 1'b1)`	Protocol check in the other direction: an answer without a request.
`if (vif.mon_cb.valid_in === 1'b1)`	The DUT sampled a transaction at this edge.
`$isunknown({...})`	The input fields of `alu_txn` are two-state, so an X on `a` would be stored as 0. Unknown inputs during a valid cycle are a testbench or connection fault; the monitor reports them instead of hiding them.
`pending = new();`	**A new object for every observed transaction.**
`pending.a = vif.mon_cb.a;` etc.	The sampled input values.

Why the monitor creates a new object for every sample

This is asked in almost every interview. A mailbox holds handles. If the monitor created one object before the loop and reused it, every entry in the mailbox would be a handle to that same object:

```
WRONG                                    RIGHT
t = new();                               forever begin
forever begin                              ...
  ...                                      t = new();
  t.a = ...; t.result = ...;               t.a = ...; t.result = ...;
  mbx.put(t);                              mbx.put(t);
end                                      end

mailbox: [h] [h] [h]                     mailbox: [h1] [h2] [h3]
          \   |   /                                 |    |    |
           v  v  v                                  v    v    v
         +---------+                             +---+ +---+ +---+
         | 1 object|   <- holds only the         | 1 | | 2 | | 3 |
         +---------+      LATEST sample          +---+ +---+ +---+
```

With the wrong version, the scoreboard may be processing transaction 1 while the monitor overwrites the same object with transaction 2. The scoreboard would then read operands from one operation and a result from another.

The bug depends on timing, so it appears and disappears as the testbench changes, which makes it very hard to find. The rule: `new()` **inside the loop, once per transaction put into a mailbox.**

Step-by-step execution for the first transaction

Same timing as in Section 3.2 (driver drives at 25, the signals change at 26):

Edge at	Sampled values (just before the edge)	Monitor action
5, 15	`rst_n` = 0	In reset: skip
25	`valid_in` = 0, `valid_out` = 0	Nothing pending, nothing valid: no action
35	`valid_in` = 1, operands of transaction 1, `valid_out` = 0	No pending. `valid_in` is 1: create `pending` with `a`, `b`, `op`, `id` = 1
45	`valid_in` = 0, `valid_out` = 1, `result` of transaction 1	Pending exists and `valid_out` is 1: store `result`, print `MON`, `put()`. No new input
55	`valid_in` = 1, operands of transaction 2, `valid_out` = 0	Create `pending` for transaction 2
65	`valid_out` = 1, `result` of transaction 2	Complete and send transaction 2

Notice edge 35: the DUT and the monitor sample the same inputs at the same edge. And edge 45: the result that the DUT registered at 35 is visible to the monitor one edge later.

> **Common mistake:** Sampling inputs and result at the same edge and putting them into one transaction. The result on the pins at edge N belongs to the inputs of edge N-1 (or is a stale value). The scoreboard then reports mismatches on almost every transaction even though the DUT is correct. A scoreboard failure pattern of "the actual result is the expected result of the previous transaction" points to this alignment mistake.

> **Common mistake:** Writing a monitor that only looks at the DUT outputs and takes the inputs from the driver's transaction. That is a half-monitor: it cannot detect driver or connection faults.

> **Verification application:** In a full environment the monitor's output goes to more than one place: the scoreboard, a functional coverage collector, and sometimes a protocol checker or a logger. In plain SystemVerilog that means one mailbox per consumer (the monitor puts a copy, or the same handle if consumers do not modify it, into each). In UVM the monitor has an *analysis port*, which broadcasts each transaction to any number of subscribers.

Interview questions

Q1. What is a monitor? A passive component that samples the DUT signals through a virtual interface at the correct clock edge, converts the pin activity into transaction objects, and sends them to the scoreboard (and coverage) through a mailbox.

Q2. What is the difference between a driver and a monitor? The driver is active: it converts transactions into signal activity on the DUT inputs. The monitor is passive: it converts signal activity on inputs and outputs back into transactions and never drives a signal.

Q3. Why does the monitor sample the DUT inputs as well? The driver already knows them. So that the checking path is independent of the stimulus path. The scoreboard then predicts from what the DUT actually received, which catches driver bugs, connection errors and timing errors, and lets monitor and scoreboard be reused where no driver exists.

Q4. At which point in time does the monitor sample? At the clocking event of `mon_cb`, with input skew `#1step`: it gets the values the signals had immediately before the rising clock edge, which are the values the DUT's flip-flops capture at that edge.

Q5. Why must the monitor create a new transaction object each time? Because the mailbox stores handles. Reusing one object means every handle in the mailbox refers to the same object, which the monitor keeps overwriting, so the scoreboard sees corrupted or duplicated data.

Q6. The DUT has a latency of one cycle. How does the monitor deal with it? It keeps the inputs it sampled at edge N in a pending transaction, and at edge N+1, when `valid_out` is sampled high, it adds the result and sends the completed transaction. For a longer or variable latency the pending transactions would be kept in a queue.

Q7. Can a monitor drive signals? No. It is passive by definition. In this project its clocking block declares only inputs, so an attempt to drive would not compile.

Q8. What checks can be placed in a monitor? Protocol checks that depend only on the signals: here, that `valid_out` appears exactly one cycle after `valid_in` and never otherwise, and that inputs are not X when valid. Data correctness is the scoreboard's job.

Q9. What happens if the monitor is slower than the DUT, or the scoreboard is slower than the monitor? The monitor must never block for longer than a clock cycle, or it will miss edges; that is why it only does zero-time work and puts into an unbounded mailbox, where `put()` never blocks. The scoreboard may lag; the mailbox buffers the transactions.

Q10. How would the monitor change for a DUT with a variable latency? Inputs would be pushed into a queue when `valid_in` is seen, and when `valid_out` is seen the oldest entry would be popped, completed and sent (assuming results come out in order). Alternatively the monitor sends input transactions and output transactions separately and the scoreboard does the matching.

4.2 The Scoreboard

In plain English

The scoreboard is the judge. For each operation the monitor reports, it works out what the result *should* have been, compares that with what the DUT actually produced, and records PASS or FAIL. At the end of the test it reports the totals.

Technical definition

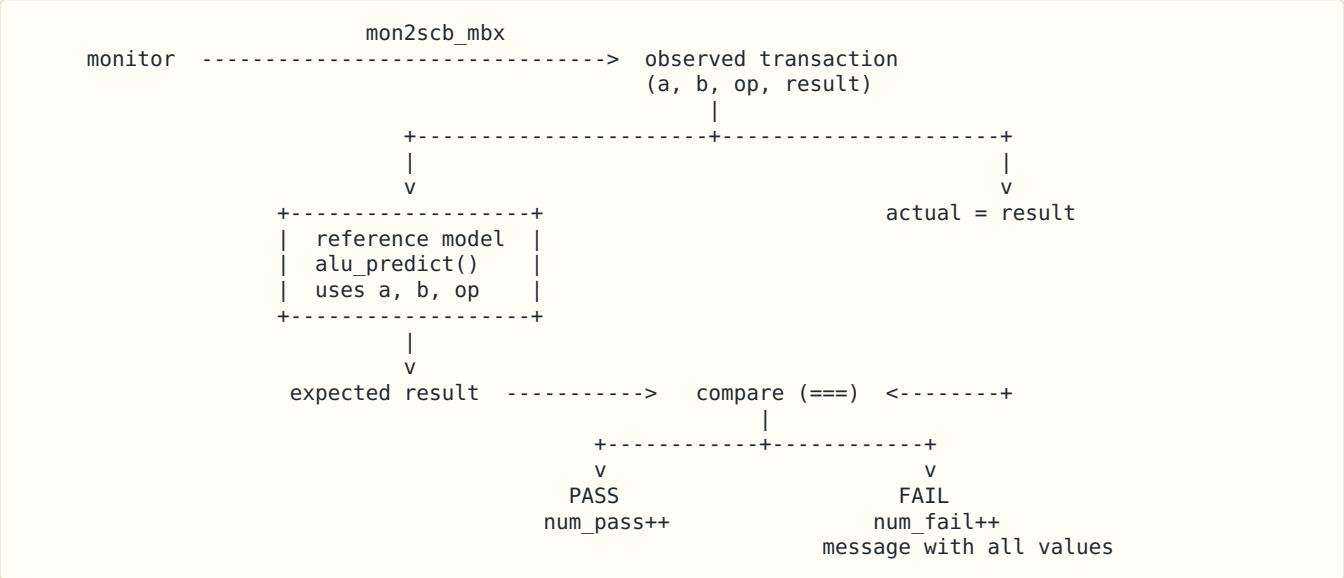
A scoreboard is a class that receives observed transactions from the monitor, obtains the expected response from a **reference model** (also called a predictor or golden model), compares expected with actual, reports each mismatch, and keeps statistics for the end-of-test verdict. It belongs to the functional layer. It is what makes the testbench **self-checking**.

Why it is needed

A testbench without a scoreboard produces waveforms and a log that a human has to inspect. That is impossible for thousands of random transactions and for the hundreds of tests in a nightly regression. With a scoreboard, every test ends in one machine-readable line: passed or failed.

Random stimulus *requires* a scoreboard. In a directed test the engineer knows the inputs in advance and can write the expected output next to them. With random inputs nobody knows the values in advance, so the expected value has to be computed at run time.

The parts of a scoreboard



Part	In this project
Input channel	`mon2scb_mbx`
Reference model	`alu_predict()`
Comparator	`alu_txn::compare()`, which uses `===` on the result
Reporting	One line per transaction, `$error` on mismatch
Statistics	`num_checked`, `num_pass`, `num_fail`, `report()`

The reference model

The reference model computes the expected result from the inputs. Three rules:

1. **It is written from the specification, not from the RTL.** If you copy the DUT's code into the model, a bug in that code is in both, and the comparison passes. The model should be an independent implementation by (ideally) a different person, written in the most obviously-correct way.
2. **It works at a higher level of abstraction.** No clocks, no pipeline registers: just "given these inputs, the output is this". The ALU's one-cycle latency does not appear in the model at all; the monitor has already dealt with it.
3. **It must model widths and signedness exactly as the specification states them** (Module 1, Sections 2.6 and 4.4). This is where most reference-model bugs are.

For a simple DUT the model is a function inside the scoreboard, as here. For a complex DUT it is a separate class, or a C/C++ model called through DPI (Module 11).

Where do the model's inputs come from?

Source of the model's inputs	Used here?	Consequence
The monitor: inputs sampled from the pins	Yes	The check is "given what the DUT received, was its output right?". Independent of generator and driver
The generator or driver, through a second mailbox	No	The check becomes "given what we meant to send...". Any fault between generator and pins corrupts the check. It also couples the scoreboard to the stimulus components, so it cannot be reused without them

Some scoreboards use both: expected transactions from the stimulus side and observed transactions from the monitor, compared field by field. That additionally verifies the driver, and it is the usual structure when the DUT transforms or reorders data (for example a router). It needs a matching scheme (in-order queues, or lookup by id). For a DUT like the ALU, where each output depends only on its own inputs, predicting from the monitored inputs is simpler and stronger.

The file

alu_scoreboard.sv

```
class alu_scoreboard;

  mailbox #(alu_txn) mon2scb_mbx;
  int unsigned       num_checked;
  int unsigned       num_pass;
  int unsigned       num_fail;

  function new(mailbox #(alu_txn) mon2scb_mbx);
    this.mon2scb_mbx = mon2scb_mbx;
  endfunction

  // Reference model: the expected 9-bit result, written from the specification
  function logic [8:0] alu_predict(alu_txn t);
    logic [8:0] r;
    case (t.op)
      ALU_ADD: r = t.a + t.b;
      ALU_SUB: r = t.a - t.b;
      ALU_AND: r = t.a & t.b;
      ALU_OR : r = t.a | t.b;
      ALU_XOR: r = t.a ^ t.b;
      ALU_NOT: r = {1'b0, ~t.a};
      ALU_SHL: r = t.a << 1;
      ALU_SHR: r = t.a >> 1;
      default: r = 'x;
    endcase
    return r;
  endfunction

  function void check_txn(alu_txn actual);
    alu_txn expected;
    expected        = actual.copy();
    expected.result = alu_predict(actual);
    if (actual.compare(expected)) begin
      num_pass++;
      $display("[%0t] SCB PASS id=%0d op=%s a=%0d b=%0d result=%0d",
               $time, actual.id, actual.op.name(), actual.a, actual.b, actual.result);
    end
    else begin
      num_fail++;
      $error("[%0t] SCB FAIL id=%0d op=%s a=%0d b=%0d actual=%0d expected=%0d",
             $time, actual.id, actual.op.name(), actual.a, actual.b,
             actual.result, expected.result);
    end
    num_checked++;
  endfunction
```

```
  task run();
    alu_txn txn;
    forever begin
      mon2scb_mbx.get(txn);
      check_txn(txn);
    end
  endtask

  function void report();
    $display("------------------------------------------------------------");
    $display("SCOREBOARD SUMMARY: checked=%0d pass=%0d fail=%0d",
             num_checked, num_pass, num_fail);
    if (num_checked == 0)   $display("TEST FAILED: nothing was checked");
    else if (num_fail == 0) $display("TEST PASSED");
    else                    $display("TEST FAILED");
    $display("------------------------------------------------------------");
  endfunction

endclass
```

Line by line

Code	Explanation
`mailbox #(alu_txn) mon2scb_mbx;`	The same mailbox the monitor puts into.
`num_checked`, `num_pass`, `num_fail`	Statistics. `num_checked` is also what the environment watches to decide that the test is over.
`function logic [8:0] alu_predict(alu_txn t);`	A function: the model takes no simulation time. It returns a 9-bit four-state value.
`logic [8:0] r;`	The result variable is 9 bits wide. Every `r = ...` assignment below is therefore evaluated in a 9-bit context: the 8-bit operands are zero-extended to 9 bits **before** the operation (Module 1, Section 4.4).
`ALU_ADD: r = t.a + t.b;`	9-bit addition; bit 8 is the carry.
`ALU_SUB: r = t.a - t.b;`	9-bit subtraction; if `b` is larger than `a` the result wraps and bit 8 is 1 (the borrow).
`ALU_AND: r = t.a & t.b;`	Both operands zero-extended; bit 8 is `0 & 0` = 0. The same for OR and XOR.
`ALU_NOT: r = {1'b0, ~t.a};`	**Not** `r = ~t.a;`. In a 9-bit context `t.a` would first be extended to `0_aaaaaaaa` and then inverted, giving a 1 in bit 8. The concatenation forces the inversion to be done on 8 bits.
`ALU_SHL: r = t.a << 1;`	The left operand of a shift takes the context width, so `a` is 9 bits wide before shifting and its old bit 7 lands in bit 8.
`ALU_SHR: r = t.a >> 1;`	Logical shift right; bit 0 is lost, zeros enter from the left.
`default: r = 'x;`	An opcode that is not one of the eight names (possible only if `op` holds X). The model answers "unknown", and the comparison then fails unless the DUT also outputs X.
`function void check_txn(alu_txn actual);`	Checks one observed transaction. A function, so it cannot block.
`expected = actual.copy();`	A separate object for the expected values, with the same inputs and id. A copy, not `expected = actual;`, which would make both handles point to one object.
`expected.result = alu_predict(actual);`	Only the result differs between expected and actual.

Code	Explanation
`if (actual.compare(expected))`	Field-by-field comparison, `===` on the result.
`$display("... SCB PASS ...")`	One line per passing transaction. In a long regression this is normally switched off or reduced; here it shows the flow.
`$error("... SCB FAIL ... actual=%0d expected=%0d")`	A mismatch message must contain everything needed to reproduce the case: time, id, operation, operands, actual and expected. `$error` raises the simulator's error count, which regression scripts read.
`num_checked++;`	Incremented last, after the message, so that a process waiting on this counter sees a fully processed transaction.
`task run();` ... `forever`	Blocks on the mailbox, checks, repeats.
`function void report();`	Called by the test after the run.
`if (num_checked == 0)`	A test in which nothing was checked must not be reported as passed.

The model above is deliberately written differently from the DUT: `<<` and `>>` instead of concatenations, plain `&` instead of `{1'b0, a & b}`. Two independent descriptions that agree are evidence that both are right.

Why expected is a copy

```
WRONG:  expected = actual;
        expected.result = alu_predict(actual);

   actual ----+
              +--> [ a b op result=300 ]     one object; the DUT's result (44) has been
   expected --+                              overwritten by the prediction; compare()
                                             compares the object with itself: always PASS

RIGHT:  expected = actual.copy();
        expected.result = alu_predict(actual);

   actual ----> [ a b op result=44  ]
   expected --> [ a b op result=300 ]        44 !== 300: FAIL reported
```

The first version turns the scoreboard into a checker that can never fail, and it does so silently. The concept behind it is handle assignment versus object copy (Module 4).

Handling multiple transactions

- The scoreboard processes transactions strictly in the order the monitor sent them; the mailbox is a FIFO.
- It may fall behind the monitor without losing anything, because the mailbox is unbounded.
- Each transaction arrives complete (inputs and result together), so no matching between separate input and output streams is needed here.
- When inputs and outputs arrive as **separate** streams (for example in a FIFO or router testbench), the scoreboard stores expected items in a queue, or in an associative array indexed by an id or address, and pops the matching one when an output arrives. Section 6.8 shows the queue version.

DUT behaviour	Scoreboard storage for expected data
One output per input, fixed latency (this ALU)	None: the monitor delivers complete transactions
One output per input, in order, variable latency (FIFO, pipeline with stalls)	Queue: `push_back` on input, `pop_front` on output

DUT behaviour	Scoreboard storage for expected data
Outputs can come out of order (router, cache, out-of-order bus)	Associative array indexed by id, tag or address
Outputs may be dropped legally	As above, plus an end-of-test rule for leftover entries

End-of-test checks

A scoreboard should fail the test not only on mismatches but also when the bookkeeping does not add up:

Check	Why
Number checked is greater than 0	A test that checks nothing proves nothing, yet "0 failures" looks like a pass
Number checked equals number sent	A lost transaction is a bug even if every received one was correct
No expected items left in the queue (when a queue is used)	The DUT failed to produce some outputs

In this project the first is in `report()`, and the second is enforced by the environment, which waits for the count to match and reports a timeout if it never does (Section 5.1).

> **Interview trap:** "My scoreboard reports no errors, so the DUT is correct." Not necessarily. Ask: how many transactions did it check? Does it use `!==`? Is the model independent of the RTL? Does the monitor sample the real pins? Does the test end only after everything is checked? A silent scoreboard is the most dangerous testbench bug, because it looks like success.

> **Common mistake:** Printing only "MISMATCH" on failure. Without the operands, operation, actual and expected values, the engineer debugging a nightly regression failure has to rerun the simulation just to find out what went wrong.

> **Common mistake:** Putting timing into the scoreboard, for example waiting a clock cycle "for the result to be ready". The scoreboard has no virtual interface and should not need one. Timing is the monitor's business.

Interview questions

Q1. What is a scoreboard? The component that checks DUT correctness automatically. It receives observed transactions from the monitor, computes the expected result with a reference model, compares the two, and reports pass or fail with statistics.

Q2. What is a reference model? An independent, higher-level implementation of the DUT's intended function, written from the specification, which computes the expected outputs for given inputs. It contains no timing.

Q3. Where does the scoreboard get the inputs for its prediction, and why? From the monitor, that is, from what was actually sampled at the DUT pins. This makes the check independent of the generator and driver, catches stimulus-path faults, and allows reuse of monitor and scoreboard at higher levels.

Q4. Why must the reference model not be copied from the RTL? A bug in the RTL would be reproduced in the model, both would give the same wrong answer, and the scoreboard would pass. Independence is what gives the comparison its value.

Q5. Which comparison operator does a scoreboard use? Case equality, `===` / `!==`, so that X or Z in the DUT output is treated as a mismatch.

Q6. What information should a scoreboard error message contain? Simulation time, transaction id, the stimulus (operation and operands), the actual result and the expected result.

Q7. How does the scoreboard handle a DUT with variable latency? It stores expected results in a queue when inputs are observed and pops the front entry when an output is observed, comparing the two. Order is preserved by the queue.

Q8. How does the scoreboard handle out-of-order outputs? It stores expected items in an associative array indexed by something that identifies the transaction (id, tag, address) and looks up the matching entry when an output arrives.

Q9. What checks are made at the end of a test? That at least one transaction was checked, that the number checked equals the number sent, that no expected items remain unmatched, and that the failure count is zero.

Q10. What is the difference between a monitor and a scoreboard? The monitor observes: it converts pins into transactions and knows the protocol but has no opinion about data correctness. The scoreboard judges: it knows the intended function but nothing about pins or timing.

Q11. Why is `expected = actual.copy()` used instead of `expected = actual`? Handle assignment would make both names refer to the same object. Writing the predicted result into `expected` would overwrite the DUT's result in `actual`, and the comparison would always pass.

Q12. Can the scoreboard contain the functional coverage? It can sample coverage, since it sees every observed transaction, but it is cleaner to put coverage in a separate component fed by the monitor, so that checking and measuring are independent.

Part 5: Environment, Test, Package and Testbench Top

5.1 The Environment

In plain English

The four components do not know about each other. Something has to create them, create the mailboxes and events they share, hand each component the right handles, start them all running at the same time, and decide when they have finished. That container is the environment.

Technical definition

The environment is a class that encapsulates the verification components for a DUT. It constructs the components and their communication channels, connects them by passing shared handles, passes down the virtual interface, starts every component's `run()` task as a parallel thread, and detects the end of the test. It contains no test-specific stimulus, so the same environment is used by every test.

Why it is a separate class

Without an environment	With an environment
Every test repeats the code that creates and connects four components	Tests construct one object
Adding a component (say a coverage collector) means editing every test	One file changes
The block's testbench cannot be instantiated inside a larger testbench	A subsystem environment contains several block environments

The file

alu_env.sv

```
class alu_env;

  virtual alu_if     vif;

  mailbox #(alu_txn) gen2drv_mbx;
  mailbox #(alu_txn) mon2scb_mbx;
  event              drv_done;
  event              gen_done;

  alu_generator      gen;
  alu_driver         drv;
  alu_monitor        mon;
  alu_scoreboard     scb;

  int unsigned       drain_timeout = 100;    // clock cycles

  function new(virtual alu_if vif);
    this.vif    = vif;
    gen2drv_mbx = new();
    mon2scb_mbx = new();
    gen = new(gen2drv_mbx, drv_done, gen_done);
```

```
    drv = new(vif, gen2drv_mbx, drv_done);
    mon = new(vif, mon2scb_mbx);
    scb = new(mon2scb_mbx);
  endfunction

  task run();
    // Start all components. Only gen.run() ever returns.
    fork
      gen.run();
      drv.run();
      mon.run();
      scb.run();
    join_any
    $display("[%0t] ENV: stimulus done, waiting for %0d results", $time, gen.num_sent);

    // Drain: wait until every transaction that was sent has been checked
    fork
      wait (scb.num_checked == gen.num_sent);
      begin
        repeat (drain_timeout) @(posedge vif.clk);
        $error("[%0t] ENV: timeout, %0d of %0d results checked",
               $time, scb.num_checked, gen.num_sent);
      end
    join_any

    disable fork;                          // stop driver, monitor, scoreboard, watchdog
  endtask

endclass
```

Line by line: construction and connection

Code	Explanation
`virtual alu_if vif;`	The environment keeps the interface handle so that it can pass it on, and so that `run()` can count clock cycles.
`mailbox #(alu_txn) gen2drv_mbx;` etc.	Handles for the two mailboxes. They are `null` until `new()`.
`event drv_done; event gen_done;`	Event variables. Unlike class handles, an event variable is created with an event object already attached; no `new()` is needed.
`alu_generator gen;` ...	Handles for the four components. Public, so that a test can reach into them (`env.gen.num_txns = 20;`).
`function new(virtual alu_if vif);`	The only thing the environment needs from outside is the interface.
`gen2drv_mbx = new();`	Creates an **unbounded** mailbox. `new(1)` would create a bounded mailbox of size 1.
`gen = new(gen2drv_mbx, drv_done, gen_done);`	Creates the generator and gives it one end of the first mailbox and the two events.
`drv = new(vif, gen2drv_mbx, drv_done);`	The driver receives the **same** mailbox handle and the **same** `drv_done` event. This single line is the connection Generator → Mailbox → Driver.
`mon = new(vif, mon2scb_mbx);`	The monitor receives the same `vif` as the driver, and one end of the second mailbox.
`scb = new(mon2scb_mbx);`	The other end: Monitor → Mailbox → Scoreboard.

The order inside the constructor matters in one respect: the mailboxes must be created **before** they are passed to the components. If `gen = new(gen2drv_mbx, ...)` came before `gen2drv_mbx = new();`, the generator would

receive `null`, the environment would then create a mailbox that only the environment knows about, and the first `put()` would fail with a null object access.

Line by line: running

Code	Explanation
`fork gen.run(); drv.run(); mon.run(); scb.run(); join_any`	Each statement in the `fork` becomes its own thread, and all four start at the same simulation time. `join_any` lets the parent continue as soon as **one** of them finishes. Driver, monitor and scoreboard are `forever` loops, so the one that finishes is always the generator. The other three keep running in the background.
`$display("... stimulus done ...")`	Printed when the generator has sent its last transaction.
second `fork` ... `join_any`	A race between two threads: "everything has been checked" and "too much time has passed".
`wait (scb.num_checked == gen.num_sent);`	Level-sensitive wait. It is re-evaluated whenever either counter changes and unblocks when the scoreboard has checked as many transactions as the generator sent.
`repeat (drain_timeout) @(posedge vif.clk);` then `$error`	The watchdog. If the DUT never produces a result, or a transaction is lost, the first thread would wait forever and the simulation would hang. The watchdog turns a hang into a clear error.
`disable fork;`	Kills every thread that this process started with `fork` and that is still running: the driver, the monitor, the scoreboard, and whichever of the two drain threads did not finish. After this the environment is quiet and `run()` returns.

Why join_any, and not join or join_none

Construct	Parent continues when	Effect if used for the four components
`fork` ... `join`	**All** threads have finished	Never: three of the four are `forever` loops. The simulation runs until someone kills it
`fork` ... `join_any`	**Any one** thread has finished	When the generator has finished. This is what is wanted
`fork` ... `join_none`	Immediately	The parent continues at once and must wait for the end of the test some other way, for example `@(gen_done);` (see below)

How the test ends

"How does your testbench know when to stop?" has several acceptable answers; a good answer names the mechanism and its weakness.

Method	How	Weakness
Fixed delay: `#10000; $finish;`	Wait a fixed time	Too short: results are cut off. Too long: wasted simulation. Breaks when the number of transactions changes
Stop when the generator finishes	`join_any`, then `$finish`	The last transactions are still in the driver, the DUT and the monitor. They are never checked

Method	How	Weakness
Generator finished **and** scoreboard has checked everything	`join_any` (or `@(gen_done)`), then `wait (scb.num_checked == gen.num_sent)`	Hangs if a transaction is lost, so it needs a timeout
The above plus a watchdog timeout	As in `alu_env::run`	None of the above; this is the robust form

The **drain time** is the interval between the last stimulus being generated and the last result being checked. Here it is one clock cycle (the DUT's latency); in a real design it can be thousands of cycles.

The role of gen_done

The generator triggers `gen_done` just before its `run()` returns. With `join_any`, the return itself tells the environment that stimulus is finished, so `alu_env::run` does not need to wait for the event. The event matters in the other common arrangement, where the components are started with `join_none`:

```
// Alternative start-up, equivalent in effect
fork
  gen.run();
  drv.run();
  mon.run();
  scb.run();
join_none
@(gen_done);                  // wait until the generator announces it has finished
```

This form is needed when the thread that finishes first is not necessarily the generator: several generators running in parallel, for instance, where the environment must wait for *all* of their done events. There is no missed-trigger race in the fragment above: threads started with `join_none` do not begin executing until the parent blocks, and the parent blocks at `@(gen_done)`, so the parent is already waiting when the generator starts.

What disable fork kills

`disable fork` terminates all the active threads that were spawned by the **calling process**. A task does not create a new process: `alu_env::run()` executes inside the `initial` block of `tb_top` that called `test.run()`. So "the calling process" is that `initial` block, and its children are the four component threads and the two drain threads. All of those that are still alive are killed. Threads belonging to other processes, such as the clock generator and the reset block in `tb_top`, are not affected.

> **Common mistake:** Putting `$finish` in the generator or the driver "when the last transaction is done". The simulation ends before the monitor and scoreboard have seen the last result, and the final transactions are never checked. A component should never end the simulation; the test (or top) does, after the environment reports that everything has drained.

> **Common mistake:** Creating two mailboxes where one is meant: `gen = new(mbx_a, ...); drv = new(mbx_b, ...);`. Both components work perfectly on their own, the generator's transactions pile up in one mailbox, and the driver waits forever on the other. Symptom: the test hangs at time 0 with only one `GEN` line printed, and (with a watchdog) ends in a timeout.

Interview questions

Q1. What is an environment? A class that contains the generator, driver, monitor and scoreboard, creates the mailboxes and events that connect them, passes the virtual interface to the driver and monitor, starts all components in parallel and controls the end of the run.

Q2. How are the generator and driver connected? The environment creates one mailbox and passes the same handle to both constructors. The generator puts transactions in; the driver gets them out.

Q3. Why are the components started with `fork` ... `join_any`? The components must run concurrently, so they are forked. Driver, monitor and scoreboard never terminate, so `join` would block forever. `join_any` returns when the generator, the only thread that ends, has finished.

Q4. After `join_any` returns, is the test over? No. The last transactions are still on their way through the driver, DUT, monitor and scoreboard. The environment waits until the scoreboard has checked as many transactions as were sent, with a timeout.

Q5. What does `disable fork` do here? It kills the still-running child threads of the current process: the three endless component threads and the unfinished drain thread. Without it they would keep running while the test prints its report.

Q6. Why does the environment need a timeout? If a transaction is lost or the DUT never responds, the condition "all checked" never becomes true and the simulation would hang. The watchdog converts the hang into an error message that names how many results were missing.

Q7. In what order must objects be created in the environment's constructor? Shared objects (mailboxes) first, then the components that receive them. Otherwise a component is given a null handle.

Q8. What is drain time? The time allowed after the last stimulus for transactions still inside the DUT and testbench to reach the scoreboard.

5.2 The Test

In plain English

The environment is the machinery; the test decides what to do with it. A test creates the environment, adjusts a few settings (how many transactions, which constraints, which generator), runs it and prints the verdict. A project has one environment and many tests.

Technical definition

A test is the top-level class of the testbench. It instantiates the environment, configures it for one scenario, starts it and reports the result. New scenarios are created by **extending** the base test and overriding its configuration, not by editing the environment or its components.

Environment versus test

	Environment	Test
Contains	Components and their connections	One environment plus scenario settings
How many per project	One per DUT	Many: one per scenario

	Environment	Test
Changes when	The testbench structure changes	A new scenario is added
Reused at higher levels	Yes	No, tests are specific to a level
Knows about constraints and counts	No (it provides defaults)	Yes

The file

alu_test.sv

```
class alu_test;

  alu_env env;
  string  name = "alu_test";

  function new(virtual alu_if vif);
    env = new(vif);
  endfunction

  // Scenario settings. Derived tests override this.
  virtual function void configure();
    env.gen.num_txns = 10;
  endfunction

  task run();
    configure();
    $display("[%0t] TEST: %s started", $time, name);
    env.run();
    env.scb.report();
  endtask

endclass
```

Line by line

Code	Explanation
`alu_env env;`	Handle to the environment.
`string name = "alu_test";`	Used in the log. Derived tests set their own name.
`function new(virtual alu_if vif);`	The test receives the interface from `tb_top`...
`env = new(vif);`	...and passes it down. This one call builds and connects the whole component tree (Section 2.2).
`virtual function void configure();`	The hook for scenario-specific settings. `virtual` is essential: `run()` calls `configure()` through a base-class handle, and the override of the object's actual class must be the one that executes (Module 4, virtual methods).
`env.gen.num_txns = 10;`	The base test: ten random transactions.
`task run();`	The execution flow of every test is the same four steps.
`configure();`	Step 1: apply the settings, after the components exist and before they run.
`env.run();`	Step 3: run until everything has been checked. This is the only statement that consumes simulation time.
`env.scb.report();`	Step 4: print the summary and verdict.

`run()` is not virtual and is not overridden: every test has the same flow, and only `configure()` differs. That keeps derived tests to a few lines (Section 6.6).

Execution flow of a test

```
tb_top:  test = new(aif)          ->  env, gen, drv, mon, scb, mailboxes built, connected
tb_top:  test.run()
            configure()           ->  scenario settings (zero time)
            env.run()
               fork ... join_any  ->  four threads start; reset; transactions flow
               drain              ->  wait for the scoreboard to catch up
               disable fork       ->  stop the endless threads
            env.scb.report()      ->  summary and verdict
tb_top:  $finish
```

Interview questions

Q1. What is the difference between a test and an environment? The environment holds the reusable structure: components and connections. The test holds what is specific to one scenario: configuration, constraints, counts. There is one environment and many tests.

Q2. How do you write a new test? Extend the base test class and override its configuration method to change counts, replace the blueprint transaction with a constrained derived transaction, or replace the generator. The environment and components are not modified.

Q3. Why is `configure()` declared virtual? The top module holds a base-class handle that may point to an object of any derived test. `run()` calls `configure()`; with a virtual method, the version belonging to the object's actual class executes. Without `virtual` the base version would always run and every test would behave like the base test.

Q4. Where is the test object created? In an `initial` block of the top module, which also passes it the interface instance.

5.3 The Testbench Package

Why a package

Classes must be declared somewhere. Declaring them at file level puts them in the compilation unit scope, whose contents depend on how the tool was invoked. Putting them in a **package** gives them a definite home: any module that needs them writes `import alu_tb_pkg::*;` and gets exactly one definition of each class. Module 11 covers packages in general.

The file

alu_tb_pkg.sv

```
package alu_tb_pkg;

  import alu_pkg::*;

  `include "alu_txn.sv"
  `include "alu_generator.sv"
  `include "alu_driver.sv"
  `include "alu_monitor.sv"
  `include "alu_scoreboard.sv"
  `include "alu_env.sv"
  `include "alu_test.sv"
  `include "alu_arith_test.sv"
  `include "alu_directed_test.sv"

endpackage
```

Line by line

Code	Explanation
`package alu_tb_pkg;`	Everything up to `endpackage` belongs to this package.
`import alu_pkg::*;`	The classes use `alu_op_e` and the opcode names. Importing here makes them visible to all the included class files.
`` `include "alu_txn.sv" ``	The preprocessor pastes the contents of the file at this point. The class is therefore declared *inside* the package, although its text lives in a separate file.
The order of the includes	A class must be declared before it is used. The transaction comes first, because every component uses it; the components come before the environment that contains them; the environment before the base test; the base test before the tests derived from it.

The last two includes are the derived tests of Sections 6.4 and 6.6.

> **Key point:** The class files are **not** listed on the compiler command line. Only `alu_tb_pkg.sv` is, and it pulls the class files in. If a class file were also compiled directly, the class would be declared twice, once outside the package and once inside, and the two would be different types. (Many teams give included class files the extension `.svh` to make this obvious. They are named `.sv` here to match the file list in the syllabus.)

If the order cannot be arranged (two classes that refer to each other), a forward declaration `typedef class alu_env;` at the top of the package tells the compiler that the name is a class that will be defined later (Module 1, Section 3.6).

A virtual interface inside a package

The classes contain `virtual alu_if vif;`, and `alu_if` is not declared in the package (an interface cannot be). This is legal: a virtual interface declaration only names the interface type, which is resolved when the design is elaborated. `alu_if.sv` must be part of the compilation, and compiling it before the package avoids warnings in some tools.

5.4 The Testbench Top: tb_top.sv

In plain English

Everything so far has been classes, which do not exist until someone calls `new()`. The testbench top is the static, module-based part: the place where the real signals live. It makes the clock, applies reset, creates one interface instance, plugs the DUT into it, and then creates the test object and tells it to run.

What belongs in the top module

Item	Why it is in the top module
Clock generation	The clock is a free-running signal needed by both the DUT and the interface before any class runs
Reset generation	Reset must be active from time 0, before the class-based components start
Interface instance	Interfaces are static design elements; a class can only *point to* one
DUT instance	The DUT is a module
DUT-to-interface connections	The only place where both are visible
Creating the test and passing it the interface	The bridge from the static world to the dynamic world
Starting the test and ending the simulation	Top-level control

What does **not** belong there: stimulus, checking, or any knowledge of what the test does.

The file

tb_top.sv

```
module tb_top;
  import alu_pkg::*;
  import alu_tb_pkg::*;

  // ---------------- clock ----------------
  logic clk = 0;
  always #5 clk = ~clk;

  // ---------------- interface and DUT ----------------
  alu_if aif (clk);

  alu dut (
    .clk       (clk),
    .rst_n     (aif.rst_n),
    .valid_in  (aif.valid_in),
    .a         (aif.a),
    .b         (aif.b),
    .op        (aif.op),
    .valid_out (aif.valid_out),
    .result    (aif.result)
  );

  // ---------------- reset ----------------
  initial begin
    aif.rst_n = 1'b0;
    repeat (2) @(negedge clk);
    aif.rst_n = 1'b1;
  end
```

```
  // ---------------- test ----------------
  initial begin
    alu_test          test;
    alu_arith_test    arith_test;
    alu_directed_test directed_test;
    string            test_name;

    if (!$value$plusargs("TEST=%s", test_name)) test_name = "alu_test";

    case (test_name)
      "alu_test"          : test = new(aif);
      "alu_arith_test"    : begin arith_test = new(aif);    test = arith_test;    end
      "alu_directed_test" : begin directed_test = new(aif); test = directed_test; end
      default             : $fatal(1, "Unknown test name '%s'", test_name);
    endcase

    test.run();
    $finish;
  end

endmodule
```

Line by line

Code	Explanation
`import alu_pkg::*;`	For the type of the `op` connection.
`import alu_tb_pkg::*;`	Makes the test classes visible.
`logic clk = 0;`	The clock starts at a known value. Initialising in the declaration creates no event at time 0, so there is no spurious edge.
`always #5 clk = ~clk;`	Period 10 time units. Rising edges at 5, 15, 25...; falling edges at 10, 20, 30...
`alu_if aif (clk);`	**The** interface instance. All testbench access to DUT signals goes through this one instance.
`alu dut (...)`	The DUT instance. Its `clk` comes straight from the clock; every other port is connected to the interface signal of the same name.
`.valid_out (aif.valid_out)`	A DUT output drives the interface variable. Each interface variable has exactly one driver: the DUT for the outputs, the clocking block `drv_cb` for the DUT inputs, the reset block for `rst_n`.
`aif.rst_n = 1'b0;`	Reset is asserted at time 0. The X → 0 transition is a negative edge, so the DUT's asynchronous reset takes effect immediately.
`repeat (2) @(negedge clk);`	Wait for two falling edges (times 10 and 20).
`aif.rst_n = 1'b1;`	Reset is released at time 20, on a falling edge, half a cycle away from the rising edges at which the DUT, driver and monitor work. That keeps the release free of races.
`alu_test test;`	A **base-class** handle. Whatever test is selected, it is run through this handle.
`alu_arith_test arith_test;` etc.	One handle per derived test, used only to construct the object.
`$value$plusargs("TEST=%s", test_name)`	Reads the command-line argument `+TEST=<name>`. It returns 0 if the argument is absent, in which case the base test is selected.
`"alu_test" : test = new(aif);`	Creates the test object. `aif`, an interface instance, is passed to a constructor argument of type `virtual alu_if`: this is the moment the physical interface is assigned to a virtual interface.
`begin arith_test = new(aif); test = arith_test; end`	Creates a derived test and stores its handle in the base handle (an upcast, always legal).

Code	Explanation
`default : $fatal(...)`	A misspelt test name must stop the run, not silently run something else.
`test.run();`	Runs the selected test. `run()` calls the virtual `configure()`, so the derived test's settings apply. Returns when everything has been checked and reported.
`$finish;`	Ends the simulation. Without it the clock generator would keep the simulation running forever.

The handles are declared at the top of the `initial` block without initialisers and assigned in the `case` branches. A variable declared in a module's `initial` block is static, and an initialiser on a static variable is executed once before simulation starts, whichever branch is later taken; writing `alu_arith_test t = new(aif);` inside a branch would therefore construct that test in every run.

One design choice: selecting the test at run time

Because the test is chosen by a plusarg, the project is compiled once and any test is run by changing the command line: `+TEST=alu_directed_test`. The `case` statement is a minimal "factory". UVM generalises it: `run_test()` reads `+UVM_TESTNAME` and creates the named test class through the UVM factory, with no `case` statement to maintain.

> **Common mistake:** Generating the clock inside a class or inside the driver. The DUT and interface need the clock from time 0 as a static signal; classes run later and may be killed with `disable fork`.

> **Common mistake:** Forgetting to pass the interface, or passing it to only part of the hierarchy. The handle that was missed remains `null`, and the first `@(vif.drv_cb)` produces a null-dereference run-time error at time 0.

> **Common mistake:** Releasing reset at the active clock edge. Whether the DUT (and the monitor's reset check) see reset as asserted or released at that edge then depends on process ordering.

> **Note:** From time 0 until the driver's first idle drive takes effect at time 6, the DUT inputs are X. The DUT is in reset, so nothing is registered. The DUT's `unique case (op)` may, in some simulators, print a "no matching case item" warning at time 0 because `op` is X. It is harmless here; it stops as soon as the driver has driven the idle opcode.

Interview questions

Q1. What does the testbench top contain? Clock generation, reset generation, the interface instance, the DUT instance connected to the interface, and an `initial` block that creates the test object, passes it the interface, runs it and finishes the simulation.

Q2. Why is the top a module and not a class? The DUT and the interface are static design elements that must be instantiated in the design hierarchy, and clock and reset must exist from time 0. Classes are dynamic and cannot instantiate modules or interfaces.

Q3. How does the physical interface reach the driver? The top module passes the interface instance to the test's constructor, whose argument is a virtual interface. The test passes it to the environment, and the environment passes it to the driver and the monitor.

Q4. How do you select which test to run without recompiling? With a plusarg read by `$value$plusargs`, and code that constructs the matching test class and assigns it to a base-class handle.

Q5. Why is the test run through a base-class handle? So that `tb_top` contains one `test.run()` call that works for every test. Polymorphism (the virtual `configure()` method) makes the selected test's behaviour execute.

Q6. Why is reset released on the falling clock edge? All sampling and driving happens around the rising edge. Changing reset half a cycle away means no component can see it change at the instant it samples.

Q7. Who calls `$finish`? The top module, after `test.run()` has returned, which happens only after the environment has drained and the scoreboard has reported.

5.5 Compiling and Running the Project

File list and compile order

Order	File	Why it comes here
1	`alu_pkg.sv`	Everything else imports it
2	`alu_if.sv`	Imports `alu_pkg`; referred to by the classes as a virtual interface
3	`alu.sv`	The DUT; imports `alu_pkg`
4	`alu_tb_pkg.sv`	Imports `alu_pkg`; includes all the class files
5	`tb_top.sv`	Imports both packages; instantiates interface and DUT

Not on the command line: `alu_txn.sv`, `alu_generator.sv`, `alu_driver.sv`, `alu_monitor.sv`, `alu_scoreboard.sv`, `alu_env.sv`, `alu_test.sv`, `alu_arith_test.sv`, `alu_directed_test.sv`. They are included by `alu_tb_pkg.sv` and only need to be in a directory where the compiler looks for include files (the current directory, or one given with `+incdir+`).

`tb_alu_direct.sv` from Part 1 is a separate testbench; do not compile it together with `tb_top.sv` unless you name the top module explicitly.

Command lines

The exact options differ between tool versions; treat these as starting points.

```
Synopsys VCS
  vcs -sverilog -full64 alu_pkg.sv alu_if.sv alu.sv alu_tb_pkg.sv tb_top.sv -o simv
  ./simv +TEST=alu_directed_test
  ./simv +TEST=alu_test +ntb_random_seed=7

Cadence Xcelium
  xrun -sv alu_pkg.sv alu_if.sv alu.sv alu_tb_pkg.sv tb_top.sv +TEST=alu_directed_test
  xrun -sv alu_pkg.sv alu_if.sv alu.sv alu_tb_pkg.sv tb_top.sv +TEST=alu_test -svseed 7

Siemens Questa
  vlog -sv alu_pkg.sv alu_if.sv alu.sv alu_tb_pkg.sv tb_top.sv
  vsim -c tb_top +TEST=alu_directed_test -do "run -all; quit"
  vsim -c tb_top +TEST=alu_test -sv_seed 7 -do "run -all; quit"
```

Option	Meaning
`+TEST=<name>`	Read by `tb_top`; selects `alu_test` (default), `alu_arith_test` or `alu_directed_test`
Seed option	Changes the random stimulus. The same seed with the same tool reproduces the same run

Running on EDA Playground

1. Choose a SystemVerilog simulator in the left panel (a commercial one; the project uses classes, clocking blocks and randomization).
2. Use the "+" next to the file tabs to create one tab for each of the fourteen project files, with exactly the file names used in this module, and paste the contents.
3. The site compiles `design.sv` first and then `testbench.sv`. Put only include lines in them:

```
// design.sv
`include "alu_pkg.sv"
`include "alu_if.sv"
`include "alu.sv"
```

```
// testbench.sv
`include "alu_tb_pkg.sv"
`include "tb_top.sv"
```

1. In the "Run Options" field enter `+TEST=alu_directed_test`, then run.

If it does not compile or run

Symptom	Likely cause
"Undefined type `alu_txn`" in a component file	A class file was compiled directly instead of through the package, or the include order in the package is wrong
"Cannot find include file"	The class files are not in the include search path; add `+incdir+<directory>`
"Undefined type `alu_op_e`"	`alu_pkg.sv` was not compiled first, or an `import alu_pkg::*;` is missing
"Null object access" at time 0	A handle was used before `new()`: usually a mailbox not created, or created after being passed on
"Null virtual interface" at time 0	The interface was not passed down to the driver or monitor
Simulation never ends	`$finish` missing, or the components were started with `join`
`ENV: timeout` error	A transaction was lost: wrong mailbox connection, monitor not seeing `valid_in` or `valid_out`, or the DUT never answering
"Unknown test name" fatal	Typing error in `+TEST=`

Part 6: Execution from Start to Finish

All the files now exist. This part follows the project as it runs: how data moves, what happens at each instant, what the log looks like, how new tests are added, and what a real failure looks like.

6.1 The Complete Communication Flow

Stimulus path

```
Generator -> Mailbox -> Driver -> Interface -> DUT
```

Step	Where	Code	What moves
1	`alu_generator::run`	`blueprint.randomize()`	Random values are chosen for `a`, `b`, `op`
2	`alu_generator::run`	`blueprint.copy()`	A new `alu_txn` object is created with those values
3	`alu_generator::send`	`gen2drv_mbx.put(txn)`	The **handle** of that object enters the mailbox
4	`alu_driver::run`	`gen2drv_mbx.get(txn)`	The handle leaves the mailbox; the driver now refers to the same object
5	`alu_driver::drive`	`vif.drv_cb.a <= txn.a;` etc.	Field values are copied onto interface signals, one time unit after a clock edge
6	`tb_top`	`.a (aif.a)` port connections	The interface signals are the wires on the DUT's input ports
7	`alu`	`always_ff @(posedge clk)`	The DUT samples the inputs at the next rising edge
8	`alu_driver::run`	`-> drv_done`	Control returns to the generator (`@(drv_done)` in `send`), which goes back to step 1

Up to step 4 the thing that moves is an object handle. From step 5 onward it is signal values. The driver is the boundary.

Checking path

```
DUT -> Interface -> Monitor -> Mailbox -> Scoreboard
```

Step	Where	Code	What moves
1	`alu`	`result <= result_nxt; valid_out <= valid_in;`	The DUT registers its outputs
2	`tb_top`	`.result (aif.result)` port connections	The DUT outputs drive the interface signals
3	`alu_if`	`clocking mon_cb`	At each rising edge the clocking block samples all signals (their pre-edge values)

Step	Where	Code	What moves
4	`alu_monitor::run`	`pending = new(); pending.a = vif.mon_cb.a;` etc.	At the edge where `valid_in` is sampled high, a new object receives the input values
5	`alu_monitor::run`	`pending.result = vif.mon_cb.result;`	One edge later the same object receives the result
6	`alu_monitor::run`	`mon2scb_mbx.put(pending)`	The handle enters the second mailbox
7	`alu_scoreboard::run`	`mon2scb_mbx.get(txn)`	The handle leaves the mailbox
8	`alu_scoreboard::check_txn`	`alu_predict`, `compare`	Expected result computed and compared; PASS or FAIL recorded
9	`alu_env::run`	`wait (scb.num_checked == gen.num_sent)`	The environment notices when the last result has been checked

Here the monitor is the boundary: signal values up to step 3, an object handle from step 4.

The two paths never share an object

The object the generator created ends its life in the driver. The object the scoreboard checks was created by the monitor from the pins. The only thing that connects the stimulus path to the checking path is the DUT itself (and the interface wires). That is deliberate: if the two paths shared an object, a mistake on one side could hide a mistake on the other.

```
generator's object:   created in gen  ->  mailbox  ->  used by driver  ->  dropped
                                                            |
                                                        pin values
                                                            |
                                                           DUT
                                                            |
                                                        pin values
                                                            |
monitor's object:                created in mon  ->  mailbox  ->  checked by scoreboard
```

An object that no handle refers to any more is reclaimed automatically by the simulator; SystemVerilog has no `delete` for objects (Module 4).

6.2 Simulation Execution Order

The threads

Once the test is running, these processes exist at the same time:

Thread	Started by	Blocks on	Ends
Clock generator	`always` in `tb_top`	`#5`	Never (until `$finish`)
Reset block	`initial` in `tb_top`	Two falling clock edges	At time 20
DUT `always_comb`, `always_ff`	The DUT	Input changes; clock and reset edges	Never
Test thread	Second `initial` in `tb_top`	`join_any`, then the drain `join_any`	Calls `$finish`

Thread	Started by	Blocks on	Ends
Generator	`fork` in `alu_env::run`	`@(drv_done)`	After the last transaction
Driver	`fork` in `alu_env::run`	Mailbox `get()`, clocking events	Killed by `disable fork`
Monitor	`fork` in `alu_env::run`	Clocking event	Killed by `disable fork`
Scoreboard	`fork` in `alu_env::run`	Mailbox `get()`	Killed by `disable fork`
Drain wait and watchdog	Second `fork` in `alu_env::run`	The counters; clock edges	One finishes, the other is killed

The phases of a run

```
BEFORE TIME 0   elaboration: tb_top, aif, dut exist. clk = 0. No class object exists yet.

TIME 0          reset block:   aif.rst_n = 0               (DUT resets asynchronously)
(zero time)     test block:    read +TEST, test = new(aif)
                                 -> BUILD + CONNECT: env, mailboxes, gen, drv, mon, scb
                               test.run()
                                 -> configure()                 (scenario settings)
                                 -> env.run(): fork four threads
                                      gen: randomize, put txn 1, wait for drv_done
                                      drv: reset(): wait for a clock edge
                                      mon: wait for a clock edge
                                      scb: wait on empty mailbox
                                    parent waits at join_any

RESET           5, 15: clock edges; drv drives idle values (visible at 6); mon skips
                20:    reset released; drv takes txn 1 from the mailbox

MAIN LOOP       every 20 time units, one transaction:
                  drive -> DUT samples -> DUT answers -> monitor sends -> scb checks

END OF STIMULUS gen has sent num_txns transactions; gen.run() returns; join_any releases
                the test thread; "ENV: stimulus done" is printed

DRAIN           the last transaction is still in the DUT; one clock later the monitor
                sends it and the scoreboard checks it; the wait condition becomes true

SHUTDOWN        disable fork kills drv, mon, scb and the watchdog; env.run() returns

REPORT          scb.report() prints the summary; tb_top calls $finish
```

What happens at time 0, in detail

Several things happen at time 0 without any time passing, and their order is worth understanding:

1. Both `initial` blocks of `tb_top` start. Their relative order is not defined, and it does not matter: one only writes `rst_n`, the other only builds objects.
2. `test = new(aif)` runs to completion (constructors are functions): every component and mailbox exists and is connected before anything runs.
3. `test.run()` → `configure()` → `env.run()` → `fork`. The four threads are created. The parent reaches `join_any` and blocks; only now do the children start executing.
4. Each child runs until it blocks. The generator randomizes, puts transaction 1 into the mailbox, prints its `GEN` line and waits for `drv_done`. The driver, monitor and scoreboard each block almost immediately.
5. Nothing more can happen at time 0. Simulation time advances to 5, the first rising clock edge.

Whichever order the simulator picks for the four children in step 4, the outcome is identical, because no child depends on another child's time-0 actions except through the mailbox, and the mailbox holds the transaction until the driver asks for it at time 20. A testbench is free of races when its behaviour does not depend on such ordering.

6.3 Cycle-by-Cycle Timeline of One Transaction

The transaction is `ALU_ADD`, `a` = 200, `b` = 100 (the first one in the directed test of the next section). Clock: rising edges at 5, 15, 25...; reset released at 20.

Time	Component	Action
0	Generator	Creates transaction 1, `put()` into `gen2drv_mbx`, prints `GEN`, waits for `drv_done`
0	Driver	In `reset()`, waiting for the first clock edge
5	Driver	Clock edge: schedules idle values on `valid_in`, `a`, `b`, `op`
6	Interface	Idle values appear
5, 15	Monitor	`rst_n` is 0: ignores these edges
20	`tb_top`	`rst_n` goes to 1
20	Driver	`wait` ends; `get()` returns transaction 1; `drive()` waits for a clock edge
25	Driver	Clock edge: schedules `valid_in` = 1, `a` = 200, `b` = 100, `op` = `ALU_ADD`
25	Monitor	Samples `valid_in` = 0, `valid_out` = 0: nothing to do
26	Interface	`valid_in` = 1 and the operands appear on the wires
26	DUT	`always_comb` recomputes `result_nxt` = 300
35	DUT	Clock edge: samples `valid_in` = 1, so `result` ← 300 and `valid_out` ← 1 (visible just after the edge)
35	Monitor	The values it sampled just before this edge: `valid_in` = 1, 200, 100, `ALU_ADD`. Creates `pending` with id 1
35	Driver	Schedules `valid_in` = 0; prints `DRV`; triggers `drv_done`; blocks in `get()`
35	Generator	Wakes; creates transaction 2; `put()`; prints `GEN`; waits again
35	Driver	`get()` returns transaction 2; `drive()` waits for a clock edge
36	Interface	`valid_in` = 0
45	DUT	Clock edge: samples `valid_in` = 0, so `valid_out` ← 0; `result` keeps 300
45	Monitor	Sampled just before this edge: `valid_out` = 1, `result` = 300. Completes transaction 1, prints `MON`, `put()` into `mon2scb_mbx`
45	Scoreboard	`get()` returns; `alu_predict` gives 300; `compare` succeeds; prints `SCB PASS`; `num_checked` = 1
45	Driver	Schedules transaction 2 on the pins (visible at 46)

From the moment the driver starts driving (25) to the scoreboard's verdict (45) is two clock cycles: one for the DUT to sample, one for the result to become visible to the monitor.

What the clocking blocks see at each edge

The same story as a table of sampled values, that is, the values just before each rising edge. This is exactly what `vif.mon_cb.*` returns after `@(vif.mon_cb)`.

```
rising edge at      5      15      25      35      45      55      65
-------------------------------------------------------------------------
rst_n               0       0       1       1       1       1       1
valid_in            x       0       0       1       0       1       0
a, b, op            x    idle    idle   txn 1   txn 1   txn 2   txn 2
valid_out           0       0       0       0       1       0       1
result              0       0       0       0     300     300   res 2
-------------------------------------------------------------------------
monitor          (reset) (reset)   -     create  send    create  send
                                         txn 1   txn 1   txn 2   txn 2
scoreboard                                       check           check
                                                 txn 1           txn 2
```

Things to notice:

- At edge 5 the inputs are still X, because the driver's first drive only appears at time 6. The DUT is in reset and the monitor is skipping, so it does not matter.
- At edge 45 the operands still show transaction 1. The driver removed `valid_in` but did not change `a`, `b` and `op`; they are irrelevant while `valid_in` is 0.
- `valid_out` at each edge equals `valid_in` at the previous edge. That is the one-cycle latency.
- `result` keeps its last value (300) at edge 55 even though `valid_out` is 0. The monitor does not look at it then.

6.4 A Directed Test, with Its Exact Log

Why a directed test in a random testbench

Random stimulus cannot be predicted, so it cannot be used to show an exact log, and a few specific cases are worth guaranteeing in every regression (the first sanity check after any change to the RTL, for example). A directed test supplies fixed transactions through the same components. Nothing in the driver, monitor, scoreboard or environment changes.

How it is done

The stimulus loop lives in `alu_generator::run`, which is virtual. A derived generator overrides `run()` with a fixed list of transactions, and a derived test puts that generator in place of the default one.

alu_directed_test.sv

```
class alu_directed_gen extends alu_generator;

  function new(mailbox #(alu_txn) gen2drv_mbx, event drv_done, event gen_done);
    super.new(gen2drv_mbx, drv_done, gen_done);
  endfunction

  task send_values(bit [7:0] a, bit [7:0] b, alu_op_e op);
    alu_txn txn;
    txn    = new();
    txn.a  = a;
    txn.b  = b;
    txn.op = op;
    send(txn);
  endtask

  virtual task run();
    send_values(8'd200, 8'd100, ALU_ADD);
    send_values(8'd5,   8'd10,  ALU_SUB);
    send_values(8'hF0,  8'h3C,  ALU_AND);
    send_values(8'hF0,  8'h3C,  ALU_OR);
    send_values(8'hF0,  8'h3C,  ALU_XOR);
    send_values(8'hF0,  8'h00,  ALU_NOT);
    send_values(8'h81,  8'h00,  ALU_SHL);
    send_values(8'h81,  8'h00,  ALU_SHR);
    -> gen_done;
  endtask

endclass

class alu_directed_test extends alu_test;

  function new(virtual alu_if vif);
    super.new(vif);
    name = "alu_directed_test";
  endfunction

  virtual function void configure();
    alu_directed_gen dgen;
    dgen    = new(env.gen2drv_mbx, env.drv_done, env.gen_done);
    env.gen = dgen;
  endfunction

endclass
```

Line by line

Code	Explanation
`class alu_directed_gen extends alu_generator;`	Inherits the mailbox handle, the events, `num_sent` and the `send()` task.
`super.new(gen2drv_mbx, drv_done, gen_done);`	The base constructor has arguments, so the derived constructor must call it explicitly and pass them on.
`task send_values(...)`	Builds one transaction with fixed values and hands it to the inherited `send()`, which numbers it, puts it in the mailbox, prints it and waits for the driver.
`virtual task run();`	Overrides the random loop of the base class.
Eight `send_values` calls	One transaction per operation.
`-> gen_done;`	The same end-of-stimulus signal as the base class.
`class alu_directed_test extends alu_test;`	A new test.
`super.new(vif);`	Builds the standard environment (including a default generator).

Code	Explanation
`name = "alu_directed_test";`	For the log.
`dgen = new(env.gen2drv_mbx, env.drv_done, env.gen_done);`	The replacement generator is connected to the **same** mailbox and events the environment already created, so it talks to the same driver.
`env.gen = dgen;`	A derived-class handle is assigned to a base-class handle. From now on `env.gen.run()` executes `alu_directed_gen::run`, because `run` is virtual, and `env.gen.num_sent` refers to the new object. The original generator object is no longer referenced.

`configure()` is called from `alu_test::run` before `env.run()`, so the replacement is in place before the threads are forked.

Expected values

id	Operation	`a`	`b`	Calculation	`result`
1	`ALU_ADD`	200	100	200 + 100 = 300 (needs bit 8)	300
2	`ALU_SUB`	5	10	5 - 10 = -5; in 9 bits 512 - 5	507
3	`ALU_AND`	240 (`F0`)	60 (`3C`)	`1111_0000 & 0011_1100` = `0011_0000`	48
4	`ALU_OR`	240	60	`1111_1100`	252
5	`ALU_XOR`	240	60	`1100_1100`	204
6	`ALU_NOT`	240	0	`~1111_0000` = `0000_1111`	15
7	`ALU_SHL`	129 (`81`)	0	`1000_0001` → `1_0000_0010`	258
8	`ALU_SHR`	129	0	`1000_0001` → `0100_0000`	64

The log

Run with `+TEST=alu_directed_test`.

EXPECTED OUTPUT

```
[0] TEST: alu_directed_test started
[0] GEN id=1 op=ALU_ADD a=200 b=100 result=x
[35] DRV id=1 op=ALU_ADD a=200 b=100 result=x
[35] GEN id=2 op=ALU_SUB a=5 b=10 result=x
[45] MON id=1 op=ALU_ADD a=200 b=100 result=300
[45] SCB PASS id=1 op=ALU_ADD a=200 b=100 result=300
[55] DRV id=2 op=ALU_SUB a=5 b=10 result=x
[55] GEN id=3 op=ALU_AND a=240 b=60 result=x
[65] MON id=2 op=ALU_SUB a=5 b=10 result=507
[65] SCB PASS id=2 op=ALU_SUB a=5 b=10 result=507
[75] DRV id=3 op=ALU_AND a=240 b=60 result=x
[75] GEN id=4 op=ALU_OR a=240 b=60 result=x
[85] MON id=3 op=ALU_AND a=240 b=60 result=48
[85] SCB PASS id=3 op=ALU_AND a=240 b=60 result=48
[95] DRV id=4 op=ALU_OR a=240 b=60 result=x
[95] GEN id=5 op=ALU_XOR a=240 b=60 result=x
[105] MON id=4 op=ALU_OR a=240 b=60 result=252
[105] SCB PASS id=4 op=ALU_OR a=240 b=60 result=252
[115] DRV id=5 op=ALU_XOR a=240 b=60 result=x
[115] GEN id=6 op=ALU_NOT a=240 b=0 result=x
[125] MON id=5 op=ALU_XOR a=240 b=60 result=204
[125] SCB PASS id=5 op=ALU_XOR a=240 b=60 result=204
[135] DRV id=6 op=ALU_NOT a=240 b=0 result=x
[135] GEN id=7 op=ALU_SHL a=129 b=0 result=x
[145] MON id=6 op=ALU_NOT a=240 b=0 result=15
[145] SCB PASS id=6 op=ALU_NOT a=240 b=0 result=15
[155] DRV id=7 op=ALU_SHL a=129 b=0 result=x
[155] GEN id=8 op=ALU_SHR a=129 b=0 result=x
[165] MON id=7 op=ALU_SHL a=129 b=0 result=258
[165] SCB PASS id=7 op=ALU_SHL a=129 b=0 result=258
[175] DRV id=8 op=ALU_SHR a=129 b=0 result=x
[175] ENV: stimulus done, waiting for 8 results
[185] MON id=8 op=ALU_SHR a=129 b=0 result=64
[185] SCB PASS id=8 op=ALU_SHR a=129 b=0 result=64
------------------------------------------------------------
SCOREBOARD SUMMARY: checked=8 pass=8 fail=0
TEST PASSED
------------------------------------------------------------
```

Reading the log

Lines	What they show
`[0] GEN id=1 ... result=x`	The generator creates the first transaction before reset is even released. `result` is X: nobody knows it yet.
`[35] DRV id=1`	Printed by the driver at the edge where the DUT sampled the transaction (driven from 26, sampled at 35). `result` is still X in the generator's object; the driver never touches it.
`[35] GEN id=2`	Same time step: `drv_done` woke the generator. The order `DRV` then `GEN` is fixed, because the first causes the second.
`[45] MON id=1 ... result=300`	One clock later the monitor has both halves. This is a *different object* from the one in the `GEN` and `DRV` lines; its values came from the pins.
`[45] SCB PASS id=1`	The scoreboard received it in the same time step. `MON` then `SCB` is also a fixed order.
Period of 20	Each transaction takes two clock cycles on the stimulus side.
`[175] ENV: stimulus done`	The last `drv_done` let the generator's `run()` return, which released `join_any`. At this moment transaction 8 has been sampled by the DUT but not yet checked.
`[185]`	Drain: one more clock, the last result is checked, the wait in the environment ends, the threads are killed, the report is printed.

Every pair of lines with the same time stamp is cause and effect within one chain of events, so the order of the lines is the same in every simulator. (A testbench in which two unrelated threads print at the same time step can legally print them in either order.)

6.5 The Random Test

Run with no `+TEST` argument (or `+TEST=alu_test`). The base test sends ten random transactions. The times and the structure of the log are identical to the directed test; the operations and operands depend on the seed and the simulator.

Sample output; operand values differ per seed and simulator:

EXPECTED OUTPUT

```
[0] TEST: alu_test started
[0] GEN id=1 op=ALU_XOR a=173 b=58 result=x
[35] DRV id=1 op=ALU_XOR a=173 b=58 result=x
[35] GEN id=2 op=ALU_SUB a=12 b=200 result=x
[45] MON id=1 op=ALU_XOR a=173 b=58 result=151
[45] SCB PASS id=1 op=ALU_XOR a=173 b=58 result=151
[55] DRV id=2 op=ALU_SUB a=12 b=200 result=x
[55] GEN id=3 op=ALU_SHL a=255 b=7 result=x
[65] MON id=2 op=ALU_SUB a=12 b=200 result=324
[65] SCB PASS id=2 op=ALU_SUB a=12 b=200 result=324
[75] DRV id=3 op=ALU_SHL a=255 b=7 result=x
[75] GEN id=4 op=ALU_AND a=0 b=91 result=x
[85] MON id=3 op=ALU_SHL a=255 b=7 result=510
[85] SCB PASS id=3 op=ALU_SHL a=255 b=7 result=510
    ... (transactions 4 to 9) ...
[215] DRV id=10 op=ALU_ADD a=255 b=255 result=x
[215] ENV: stimulus done, waiting for 10 results
[225] MON id=10 op=ALU_ADD a=255 b=255 result=510
[225] SCB PASS id=10 op=ALU_ADD a=255 b=255 result=510
------------------------------------------------------------
SCOREBOARD SUMMARY: checked=10 pass=10 fail=0
TEST PASSED
------------------------------------------------------------
```

Check the arithmetic of a sample line yourself; it is good practice for reading real logs: 173 is `1010_1101`, 58 is `0011_1010`, XOR gives `1001_0111` = 151. 12 - 200 = -188, and 512 - 188 = 324. 255 shifted left is `1_1111_1110` = 510.

Nobody computed these expected values in advance. The reference model did, at run time, for whatever the generator produced. That is the point of the architecture.

> **Key point:** A failing random test is only useful if it can be reproduced. Always record the seed (simulators print it, or it is given on the command line), and rerun with the same seed to get the same stimulus.

6.6 Adding a Test Without Touching the Components

The requirement

"Stress the adder and subtractor: send 20 transactions that are only `ALU_ADD` or `ALU_SUB`."

The solution: extend the transaction and the test

alu_arith_test.sv

```
class alu_arith_txn extends alu_txn;

  constraint arith_c {
    op inside {ALU_ADD, ALU_SUB};
  }

endclass

class alu_arith_test extends alu_test;

  function new(virtual alu_if vif);
    super.new(vif);
    name = "alu_arith_test";
  endfunction

  virtual function void configure();
    alu_arith_txn arith_txn;
    arith_txn         = new();
    env.gen.blueprint = arith_txn;
    env.gen.num_txns  = 20;
  endfunction

endclass
```

Line by line

Code	Explanation
`class alu_arith_txn extends alu_txn;`	A transaction with everything `alu_txn` has, plus one more constraint.
`constraint arith_c { op inside {ALU_ADD, ALU_SUB}; }`	Added to the inherited `operand_c`. Both must hold when the object is randomized.
`env.gen.blueprint = arith_txn;`	The generator's blueprint handle (type `alu_txn`) now refers to an `alu_arith_txn` object.
`env.gen.num_txns = 20;`	A scenario setting.

Why this works

```
   generator code (unchanged):      blueprint.randomize()
                                         |
                                         v
   blueprint handle type : alu_txn       randomize() acts on the OBJECT, and the object
   object it refers to   : alu_arith_txn is an alu_arith_txn, so operand_c AND arith_c
                                         are both applied

                                    send(blueprint.copy())
                                         |
                                         v
                                    copy() creates a plain alu_txn carrying the
                                    already-constrained values of a, b, op
```

- `randomize()` always uses the constraints of the object's actual class, whatever the type of the handle through which it is called (Module 5).
- The copy that travels through the mailbox is a plain `alu_txn`. That is fine: the derived class adds only a constraint, which has done its work by the time the values are copied. A derived transaction that added **fields** would also override `copy()`, which is why `copy()` is virtual.

What changed, and what did not

File	Changed?
`alu_arith_test.sv`	New
`alu_tb_pkg.sv`	One `` `include `` line
`tb_top.sv`	One `case` branch (with UVM's factory, not even that)
`alu_txn.sv`, `alu_generator.sv`, `alu_driver.sv`, `alu_monitor.sv`, `alu_scoreboard.sv`, `alu_env.sv`, `alu_test.sv`	**No**

The scoreboard checks the new stimulus without modification, because it predicts from whatever the monitor reports.

Sample output with `+TEST=alu_arith_test`; operand values differ per seed and simulator:

EXPECTED OUTPUT

```
[0] TEST: alu_arith_test started
[0] GEN id=1 op=ALU_ADD a=37 b=255 result=x
[35] DRV id=1 op=ALU_ADD a=37 b=255 result=x
[35] GEN id=2 op=ALU_SUB a=0 b=142 result=x
[45] MON id=1 op=ALU_ADD a=37 b=255 result=292
[45] SCB PASS id=1 op=ALU_ADD a=37 b=255 result=292
[55] DRV id=2 op=ALU_SUB a=0 b=142 result=x
[55] GEN id=3 op=ALU_ADD a=255 b=255 result=x
[65] MON id=2 op=ALU_SUB a=0 b=142 result=370
[65] SCB PASS id=2 op=ALU_SUB a=0 b=142 result=370
    ... (transactions 3 to 19) ...
[415] DRV id=20 op=ALU_SUB a=90 b=17 result=x
[415] ENV: stimulus done, waiting for 20 results
[425] MON id=20 op=ALU_SUB a=90 b=17 result=73
[425] SCB PASS id=20 op=ALU_SUB a=90 b=17 result=73
------------------------------------------------------------
SCOREBOARD SUMMARY: checked=20 pass=20 fail=0
TEST PASSED
------------------------------------------------------------
```

Three ways a test can change the stimulus

Technique	What the test does	Used in
Change a knob	`env.gen.num_txns = 20;`	Every test
Replace the blueprint	`env.gen.blueprint = <derived transaction>;`	`alu_arith_test`
Replace the generator	`env.gen = <derived generator>;`	`alu_directed_test`

All three rely on the same two OOP facts from Module 4: a base-class handle can refer to a derived-class object, and a virtual method (or `randomize()`) executes according to the object, not the handle.

> **Interview trap:** "To add a test for only ADD operations, I would add an `if` in the generator." That works once. After twenty tests the generator is a tangle of test-specific branches that every test depends on. The interviewer wants to hear: extend the transaction with a constraint, extend the test, set the blueprint. Components stay closed to modification and open to extension.

6.7 A Seeded Bug: What a Failure Looks Like

A testbench that has never reported a failure has not been shown to work. A standard practice is to put a deliberate bug into the DUT and confirm that the testbench finds it. It also teaches how to read a failure.

The bug

In `alu.sv`, change the addition line to:

```
ALU_ADD: result_nxt = {1'b0, a + b};      // BUG: the carry is lost
```

It looks harmless, and it is a real kind of mistake. Inside a concatenation each operand is self-determined, so `a + b` is computed in 8 bits and the carry is discarded before the zero is placed in front (Module 1, Section 4.4).

The directed test now prints

The message text is shown; `$error` also adds a simulator-specific prefix with severity, file and line.

EXPECTED OUTPUT

```
[0] TEST: alu_directed_test started
[0] GEN id=1 op=ALU_ADD a=200 b=100 result=x
[35] DRV id=1 op=ALU_ADD a=200 b=100 result=x
[35] GEN id=2 op=ALU_SUB a=5 b=10 result=x
[45] MON id=1 op=ALU_ADD a=200 b=100 result=44
[45] SCB FAIL id=1 op=ALU_ADD a=200 b=100 actual=44 expected=300
[55] DRV id=2 op=ALU_SUB a=5 b=10 result=x
    ... (transactions 2 to 8 pass exactly as before) ...
------------------------------------------------------------
SCOREBOARD SUMMARY: checked=8 pass=7 fail=1
TEST FAILED
------------------------------------------------------------
```

Localising the bug from the log alone

Question	Evidence in the log	Conclusion
Did the stimulus reach the DUT correctly?	`GEN`, `DRV` and `MON` lines for id 1 all show `ALU_ADD`, 200, 100	Yes. Generator, driver and wiring are fine: the monitor saw on the pins what the generator intended
What did the DUT output?	`MON ... result=44`	44, sampled from the pins
What should it have been?	`expected=300`	200 + 100
How do they differ?	300 - 44 = 256 = 2^8	Exactly bit 8 is missing; the low 8 bits are right
Which operations fail?	Only id 1; the other seven pass	Only `ALU_ADD`
Is the model or the DUT wrong?	The specification says bit 8 of an addition is the carry	The model agrees with the specification; the DUT is wrong

Conclusion before opening a waveform: *the ALU drops the carry out of bit 7 for `ALU_ADD`*. That points at one line of RTL.

With the random arithmetic test the same bug shows a second pattern. Sample output; operand values differ per seed and simulator:

EXPECTED OUTPUT

```
[45] SCB FAIL id=1 op=ALU_ADD a=37 b=255 actual=36 expected=292
[65] SCB PASS id=2 op=ALU_SUB a=0 b=142 result=370
[85] SCB FAIL id=3 op=ALU_ADD a=255 b=255 actual=254 expected=510
[105] SCB PASS id=4 op=ALU_ADD a=3 b=9 result=12
```

Additions fail only when the sum exceeds 255 (37 + 255, 255 + 255), and pass otherwise (3 + 9). The pattern across many random transactions narrows the cause faster than any single directed case. It also shows why the constraint `operand_c` makes 255 a frequent operand: without it, a small random sample might contain few overflowing additions.

When the testbench is the one that is wrong

Suppose instead the reference model had been written as `ALU_NOT: r = ~t.a;`. The directed test would print:

EXPECTED OUTPUT

```
[145] SCB FAIL id=6 op=ALU_NOT a=240 b=0 actual=15 expected=271
```

The same reasoning applies, with the opposite conclusion. Expected minus actual is 256 again, but now the specification says the result of NOT is `~a`, an 8-bit quantity, so bit 8 must be 0. The DUT's 15 is right and the model's 271 is wrong: in a 9-bit context `~t.a` inverts the extended zero in bit 8 as well.

> **Key point:** A scoreboard mismatch means "DUT and model disagree". It does not say which one is wrong. The specification decides. A noticeable fraction of the failures in a new testbench are testbench bugs, and an engineer who assumes the DUT is always the guilty one wastes the designer's time.

A debugging order that works

1. Read the first failure, not the last. Later failures are often consequences.
2. From the `FAIL` line: which operation, which operands, how do actual and expected differ (one bit? off by one? shifted? the previous transaction's value?).
3. Compare the `GEN` / `DRV` lines with the `MON` line for the same id. If they differ, the problem is on the stimulus path (driver, interface, connections), not in the DUT's function.
4. Compute the correct answer by hand from the specification. That tells you whether DUT or model is wrong.
5. Only then open the waveform, at the time given in the log, to see why.
6. Rerun with the same seed after the fix, then run the whole regression.

6.8 The Same Structure for a Different DUT: a Synchronous FIFO

The architecture does not depend on the ALU. To show what is general and what is specific, here is how the same structure maps onto an 8-bit synchronous FIFO, `sync_fifo`, with ports `clk`, `rst_n`, `wr_en`, `rd_en`, `din[7:0]`, `dout[7:0]`, `full` and `empty`. The code below is a sketch (fragments, not a complete project); it assumes that `dout` is registered, so that the data for a read accepted at one clock edge is visible at the next.

What stays and what changes

Item	ALU project	FIFO project	Same structure?
Top module	Clock, reset, `alu_if`, `alu`	Clock, reset, `fifo_if`, `sync_fifo`	Yes
Interface	`alu_if` with `drv_cb`, `mon_cb`	`fifo_if` with `drv_cb`, `mon_cb`	Yes
Transaction	`a`, `b`, `op`, `result`	`kind` (write or read), `data`	Fields differ
Generator	Random transactions via blueprint	The same code, different transaction class	Yes
Driver	`valid_in` for one cycle	`wr_en` or `rd_en` for one cycle	Protocol differs
Monitor	Pairs inputs with the result one cycle later	Reports accepted writes; reports accepted reads with the data one cycle later	Protocol differs
Scoreboard	Stateless model `alu_predict`	**Stateful** model: a queue	Model differs
Environment, test	As shown	Identical in shape	Yes
End of test	Checked count equals sent count	Same, plus "model queue matches the DUT's fill level"	Yes

The important difference is in the scoreboard. The ALU's output depends only on the current inputs, so its model is a pure function. A FIFO's output depends on history: what comes out now is what went in earlier. Its reference model must **remember**, and the natural SystemVerilog structure for "first in, first out" is a queue (Module 2).

Transaction and scoreboard sketch

```
typedef enum bit {FIFO_WRITE, FIFO_READ} fifo_kind_e;

class fifo_txn;
  rand fifo_kind_e kind;
  rand bit [7:0]   data;       // WRITE: the value to write
  logic [7:0]      rdata;      // READ : the value the monitor saw on dout
endclass

class fifo_scoreboard;

  mailbox #(fifo_txn) mon2scb_mbx;
  bit [7:0]           model_q [$];       // reference model: the expected FIFO contents
  int unsigned        num_checked;
  int unsigned        num_fail;

  function new(mailbox #(fifo_txn) mon2scb_mbx);
    this.mon2scb_mbx = mon2scb_mbx;
  endfunction

  task run();
    fifo_txn  txn;
    bit [7:0] expected_data;
    forever begin
      mon2scb_mbx.get(txn);
      case (txn.kind)
        FIFO_WRITE: model_q.push_back(txn.data);          // remember what went in
        FIFO_READ : begin
          if (model_q.size() == 0) begin
            num_fail++;
            $error("[%0t] SCB FAIL: DUT produced data but the model is empty", $time);
          end
          else begin
```

```
            expected_data = model_q.pop_front();          // the oldest value must come out
            if (txn.rdata !== expected_data) begin
              num_fail++;
              $error("[%0t] SCB FAIL: read %0h, expected %0h",
                     $time, txn.rdata, expected_data);
            end
          end
          num_checked++;
        end
      endcase
    end
  endtask

endclass
```

How the pieces behave

Component	FIFO behaviour
Monitor	At each clock edge: if it sampled `wr_en` = 1 and `full` = 0, it sends a `FIFO_WRITE` transaction with `din`. If it sampled `rd_en` = 1 and `empty` = 0, it notes a pending read and, at the next edge, sends a `FIFO_READ` transaction with `dout`. Writes when full and reads when empty are not sent, because the DUT ignores them
Scoreboard, on a write	`push_back` the data onto the model queue
Scoreboard, on a read	`pop_front` the oldest value and compare with what the DUT output
Flags	`model_q.size() == 0` should agree with `empty`, and `model_q.size() == DEPTH` with `full`. These can be checked in the scoreboard (if the monitor reports the flags) or by assertions (Module 10)
End of test	The number of reads checked equals the number of accepted reads, and the model queue holds exactly the data still inside the DUT

Simultaneous read and write in the same cycle is the classic FIFO corner case. The monitor sends both transactions for that edge; sending the write first is correct for a FIFO whose simultaneous read returns the previously oldest entry, as long as the model was not empty.

> **How to say it in an interview:** "For a FIFO I would keep the same generator, driver, monitor, scoreboard, environment structure. The transaction has a kind (write or read) and data. The scoreboard's reference model is a SystemVerilog queue: on every write the monitor reports, I `push_back` the data; on every read, I `pop_front` and compare with `dout`. I would also check `full` and `empty` against the queue size, and at the end of the test check that nothing unexpected is left."

6.9 The Classic Design Questions, Answered in One Place

Question	Short answer	Section
How does the test end?	Generator finishes (`join_any`), then wait until the scoreboard has checked as many transactions as were sent, guarded by a timeout; then `disable fork`, report, `$finish`	5.1
Why does the monitor create a new object per transaction?	The mailbox stores handles; reusing one object would overwrite transactions still waiting to be checked	4.1

Question	Short answer	Section
Why does the scoreboard predict from the monitor's inputs and not the generator's?	To check what the DUT actually received; it keeps the checking path independent of the stimulus path and reusable without it	4.2
Where is the reference model?	In the scoreboard (`alu_predict`), written from the specification, with no timing	4.2
How is the one-cycle latency handled?	In the monitor: inputs sampled at edge N are held in a pending transaction and completed with the result sampled at edge N+1	4.1
What happens during reset?	`tb_top` drives `rst_n` ; the driver drives idle values and waits for release; the monitor ignores edges while reset is active	3.2, 4.1, 5.4
How is PASS or FAIL decided?	Scoreboard counters; fail if any mismatch, if nothing was checked, or if the drain times out	4.2, 5.1
How do you add a test?	Extend the test; change knobs, replace the blueprint or replace the generator in `configure()`	6.6
How does a class reach the pins?	Virtual interface handle passed from `tb_top` through test and environment to driver and monitor	2.4, 5.4
How do you avoid races between testbench and DUT?	Clocking blocks: sample just before the edge, drive after it; reset changed on the opposite edge	2.4
Why a mailbox and not a direct method call?	It decouples the two sides, buffers items and blocks the receiver when empty	3.1
Why a copy of the blueprint?	Each transaction in the mailbox must be a separate object	3.1

Interview questions

Q1. Explain the flow of one transaction through your testbench. The generator randomizes a transaction and puts a copy in a mailbox. The driver gets it, waits for a clock edge and drives the fields onto the interface through its clocking block, with `valid_in` high for one cycle. The DUT samples the inputs at the next rising edge and produces the result one cycle later. The monitor, sampling through its own clocking block, captures the inputs at the edge the DUT sampled them and the result one edge later, puts both in a new transaction and sends it through a second mailbox to the scoreboard. The scoreboard computes the expected result with its reference model, compares using case equality and records pass or fail.

Q2. How many objects does one operation create, and who creates them? Two that matter: the generator's copy of the blueprint, which carries the stimulus to the driver, and the monitor's object, which carries the observation to the scoreboard. (The scoreboard also makes a temporary copy to hold the expected values.)

Q3. In the log, the `GEN` line shows `result=x` and the `MON` line shows `result=300`. Why? They are different objects. The generator's transaction never receives a result: `result` keeps its default X. The monitor's transaction is built from the pins, including the DUT's result.

Q4. What exactly happens at the end of the test? The generator's `run()` returns, releasing `join_any`. The environment then waits for `scb.num_checked == gen.num_sent`, in parallel with a timeout. When the condition is met, `disable fork` kills the driver, monitor, scoreboard and watchdog threads; the test prints the scoreboard report; and the top module calls `$finish`.

Q5. You need a test that sends only shift operations. What do you write? A transaction class derived from `alu_txn` with a constraint `op inside {ALU_SHL, ALU_SHR}`, and a test derived from `alu_test` whose `configure()` assigns an object of that class to `env.gen.blueprint`. No component changes.

Q6. Why can the generator randomize a derived transaction through a base-class handle? `randomize()` is a built-in virtual method: it solves the constraints of the actual object the handle refers to, including those added by the derived class.

Q7. The scoreboard reports a mismatch. How do you decide whether the DUT or the testbench is wrong? Compare the stimulus lines (`GEN` / `DRV`) with the monitor line to rule out the stimulus path; then compute the correct answer by hand from the specification and see whether the DUT's actual value or the model's expected value matches it.

Q8. Why is it useful to inject a bug into the DUT deliberately? To prove that the testbench can fail. A testbench with a broken scoreboard, a monitor that samples nothing, or a test that ends too early also reports "no errors".

Q9. How would the scoreboard differ for a FIFO? The reference model must hold state. A queue stores the data written; each read pops the front of the queue and compares it with the DUT's output. Flags are checked against the queue size.

Q10. Two log lines have the same time stamp. Can you rely on their order? Only if one causes the other (for example the monitor's `put()` waking the scoreboard). Lines printed by independent threads in the same time step can appear in either order, and a testbench must not depend on that order.

Q11. If the DUT were changed to a two-cycle latency, what would you change? The monitor's pairing logic (hold pending inputs for two edges, most simply with a queue of pending transactions). The driver, generator, scoreboard, environment and tests would not change; the reference model has no notion of latency.

Q12. What would you add to this testbench next? Functional coverage on the monitored transactions (operations, corner operands, crosses), assertions in the interface for the `valid_in` / `valid_out` protocol and X checks, back-to-back transactions without the idle cycle, reset in the middle of a test, and more tests built from constrained transactions.

Part 7: Module 9 Revision and Practice

7.1 Revision Summary

The architecture on one page

```
Stimulus path :  Generator -> Mailbox -> Driver -> Interface -> DUT
Checking path :  DUT -> Interface -> Monitor -> Mailbox -> Scoreboard

tb_top (module):  clock, reset, alu_if aif, alu dut, test = new(aif), test.run(), $finish
   test  (class):  env = new(vif); configure(); env.run(); env.scb.report()
      env (class): mailboxes, events, gen, drv, mon, scb;
                   fork ... join_any; drain; disable fork
```

Each component in one line

Component	Level	Input	Output	Has `vif`?	Ends by itself?
Transaction	Data			No	n/a
Generator	Transaction	Constraints, blueprint	Transactions into `gen2drv_mbx`	No	Yes
Driver	Transaction → pins	Transactions from `gen2drv_mbx`	Pin activity through `drv_cb`	Yes	No
Monitor	Pins → transaction	Samples through `mon_cb`	Transactions into `mon2scb_mbx`	Yes	No
Scoreboard	Transaction	Transactions from `mon2scb_mbx`	PASS/FAIL, counters, report	No	No
Environment	Structure	`vif`	Running components	Passes it on	When drained
Test	Scenario	`vif`, plusargs	Configuration, verdict	Passes it on	After report
Top	Static		Clock, reset, instances	Owns the instance	Calls `$finish`

Who knows what

Knowledge	Generator	Driver	Monitor	Scoreboard	Test
What values are legal (constraints)	Via the transaction				Yes
Pin names and clock edges		Yes	Yes		
DUT latency		Partly	Yes		
What the correct result is				Yes	
How many transactions	Yes			Counts	Yes

Rules to remember

Structure

- Only the driver and the monitor touch signals, and only through a virtual interface and clocking blocks.
- Components never call each other's methods. They share mailboxes and events created once in the environment.
- Create shared objects before passing them on. A handle that was never assigned is `null`.
- Class files are included in one package; only the package is compiled.

Objects and handles

- A mailbox carries handles. Every transaction put into a mailbox must be its own object: `copy()` in the generator, `new()` inside the loop in the monitor.
- `expected = actual.copy()`, never `expected = actual`.
- Stimulus fields are `rand bit`; fields sampled from DUT outputs are `logic`.

Timing

- A clocking block input is the value just before the edge; a clocking block output appears a skew after the edge.
- The driver aligns to a clocking event before every drive.
- The monitor pairs the inputs of edge N with the outputs of edge N+1 (the DUT's latency).
- Reset is driven by the top module, away from the active clock edge. The driver idles the bus and waits; the monitor skips.

Checking

- The reference model is written from the specification, has no timing, and uses exact widths.
- The scoreboard predicts from what the monitor observed.
- Compare with `===`. Print time, id, stimulus, actual and expected on a mismatch.
- Fail the test if nothing was checked or if not everything sent was checked.

Control

- `fork` ... `join_any` starts the components; the generator is the thread that finishes.
- End of test = generator done **and** scoreboard caught up, with a timeout; then `disable fork`.
- Only the top module calls `$finish`.
- New tests extend the base test and change knobs, the blueprint or the generator. Components are not edited.

Direct versus layered, in four lines

	Direct	Layered
Stimulus and expected values	Hand-written	Constrained-random; expected from a reference model
Structure	One module	Reusable class components plus one top module
Cost	Low at first, grows with every test	High at first, low per additional test
Use	Small blocks, bring-up	Industry standard; basis of UVM

7.2 Interview Questions and Answers

Basic level

Q1. Draw and explain a layered testbench. Draw the diagram of Section 1.3: test containing environment; environment containing generator, driver, monitor and scoreboard; generator to driver through a mailbox; monitor to scoreboard through a mailbox; driver and monitor connected to the interface through a virtual interface; interface connected to the DUT inside the top module. Then give each component's one-line job.

Q2. List the components of a layered testbench. Transaction, generator, driver, monitor, scoreboard, environment, test, interface, and the testbench top module. A coverage collector is usually added.

Q3. What is the role of the generator? To create and randomize transactions and send them to the driver.

Q4. What is the role of the driver? To convert transactions into pin-level activity on the DUT inputs according to the protocol.

Q5. What is the role of the monitor? To sample the DUT pins passively and convert the activity back into transactions for the scoreboard and coverage.

Q6. What is the role of the scoreboard? To compare the DUT's actual outputs with the expected outputs from a reference model and report pass or fail.

Q7. What connects the generator to the driver? A mailbox of transaction handles, created in the environment and shared by both.

Q8. What connects the driver to the DUT? A virtual interface: a handle in the driver that points to the interface instance in the top module, whose signals are wired to the DUT ports.

Q9. Which components have a virtual interface handle? The driver and the monitor. The environment and test hold one only to pass it down.

Q10. What is a self-checking testbench? A testbench that determines pass or fail by itself, by comparing DUT outputs with expected values computed automatically, with no manual inspection of waveforms.

Q11. Why is a mailbox used instead of a queue between generator and driver? A mailbox is built for communication between threads: `get()` blocks while it is empty and `put()` blocks while a bounded mailbox is full, and its operations are atomic. A queue has no blocking; the driver would have to poll it, and concurrent access would need extra protection.

Q12. What is the top module responsible for? Clock and reset generation, instantiating the interface and the DUT and connecting them, creating the test with the interface handle, running it, and ending the simulation.

Q13. In which order are the testbench objects created? The top module creates the test; the test's constructor creates the environment; the environment's constructor creates the mailboxes and then the generator, driver, monitor and scoreboard.

Q14. What is the difference between `fork`…`join`, `join_any` and `join_none`? `join` waits for all forked threads, `join_any` for the first one to finish, `join_none` for none. The environment uses `join_any` because only the generator finishes.

Q15. Which signals does the driver drive, and which does the monitor sample? The driver drives the DUT inputs (`valid_in`, `a`, `b`, `op`). The monitor samples inputs and outputs (`valid_in`, `a`, `b`, `op`, `valid_out`, `result`).

Intermediate level

Q16. Why are the generator and driver separate components? What to send and how to send it change for different reasons. Separating them lets many generators (scenarios) share one driver, and lets the driver change with the protocol without affecting the scenarios.

Q17. Why are the driver and monitor separate, when the driver could also sample the outputs? The monitor must work without the driver: when the block is integrated into a larger design the driver disappears, but the monitor and scoreboard are still needed. Separation also keeps the checking path independent of the stimulus path.

Q18. Why are the monitor and scoreboard separate? The monitor depends on the protocol; the scoreboard depends on the function. Either can change without the other. The monitor's transactions also feed coverage and other subscribers.

Q19. How does your testbench avoid race conditions with the DUT? The driver and monitor use clocking blocks. Inputs are sampled with `#1step` skew (the value before the clock edge) and outputs are driven a skew after the edge, so the testbench never reads or writes a signal at the same moment as the DUT's clocked logic. Reset is changed on the opposite clock edge.

Q20. What is the blueprint pattern? The generator randomizes a template object that a test can replace with a derived-class object, and sends copies of it. It lets tests change constraints without modifying the generator.

Q21. How do the generator and driver stay synchronised? The generator waits for the event `drv_done` after each `put()`, and the driver triggers the event when the transaction has been applied. Alternatively a mailbox of size 1 provides back-pressure.

Q22. What would happen if the environment used `fork`…`join`? It would never return, because driver, monitor and scoreboard run `forever`. The simulation would hang until a timeout.

Q23. What would happen if the test called `$finish` as soon as the generator finished? The transactions still in the driver, the DUT and the monitor would never be checked. The test would report a pass on fewer transactions than it sent.

Q24. How do you make sure every generated transaction was checked? Compare counters at the end: the number the scoreboard checked must equal the number the generator sent. The environment waits for that equality with a timeout, and the report fails if nothing was checked.

Q25. Where are the classes declared, and how does the top module see them? In a package that includes the class files. The top module imports the package.

Q26. Why can a class not instantiate the interface itself? An interface is a static design element that must exist in the elaborated hierarchy and be wired to the DUT. Classes are created at run time. A class can only hold a virtual interface handle that refers to an existing instance.

Q27. How is the DUT reset in this testbench, and how do the components react? The top module drives `rst_n` low from time 0 and releases it after two falling clock edges. The driver first drives idle values and waits for `rst_n` to be 1. The monitor discards anything it sees while `rst_n` is not 1.

Q28. What does the scoreboard do if the DUT output is X? The comparison uses `===`, so X does not match the expected value and a failure is reported.

Q29. How do you run a different test without recompiling? The top module reads a plusarg (`+TEST=name`), constructs the corresponding test class and runs it through a base-class handle.

Q30. What is the UVM equivalent of each component? Transaction: `uvm_sequence_item`. Generator: `uvm_sequence` running on a `uvm_sequencer`. Driver: `uvm_driver`. Monitor: `uvm_monitor`. Driver, sequencer and monitor together: `uvm_agent`. Scoreboard: `uvm_scoreboard`. Environment: `uvm_env`. Test: `uvm_test`. Mailboxes: TLM ports and FIFOs. Passing the virtual interface: `uvm_config_db`.

Advanced and scenario-based

Q31. The simulation hangs at time 0 after printing one `GEN` line. What do you check? The generator is waiting for `drv_done`, so the driver never completed a transaction. Check, in order: is the driver's mailbox the same object as the generator's; is `vif` assigned; is the clock running; was reset released (the driver waits for it); does the driver trigger the same event object the generator waits on.

Q32. Every transaction fails, and in each failure the actual result equals the expected result of the previous transaction. What is wrong? The monitor is pairing inputs and outputs from the same clock edge. The output at an edge belongs to the inputs sampled one edge earlier. Fix the monitor's latency alignment.

Q33. The test passes, but you later find that the DUT's `result` was X throughout. Name three testbench faults that could cause this. The scoreboard compares with `!=` / `==`; the transaction's `result` field is two-state, so X became 0 and happened to match; the scoreboard checked zero transactions and the report did not treat that as a failure; or `expected` and `actual` were the same object.

Q34. The scoreboard sees duplicate transactions: the same values several times, and some values never. Why? The monitor (or generator) is putting the same object into the mailbox repeatedly instead of creating a new one each time, so all handles refer to one object holding the latest values.

Q35. A test passes with one seed and fails with another. Is that a testbench problem? Not necessarily; that is what random testing is for. The failing seed reached a scenario the passing one did not. Rerun with the failing seed to reproduce it, and debug as usual. It is a testbench problem only if the *same* seed gives different results between runs, which indicates a race.

Q36. The same test and seed pass on one simulator and fail on another. What do you suspect? A race condition: behaviour that depends on the order in which the simulator executes processes in the same time step. Typical causes are driving or sampling at the clock edge without a clocking block, an event triggered before its waiter is waiting, or two threads writing a shared variable. Different random number generators between simulators also change the stimulus, so first confirm that the failing scenario is the same.

Q37. How would you verify a DUT with two input interfaces and one output interface? One agent-like group (generator, driver, monitor) per input interface and a monitor on the output interface, all inside one environment. The scoreboard receives expected data from the input monitors through one mailbox each and actual data from the output monitor, with a reference model that combines the inputs.

Q38. Your block-level environment must be reused at subsystem level, where real logic drives the ALU. What changes? The generator and driver are not created (the environment gets a configuration flag for "passive" mode). The monitor and scoreboard are created and connected exactly as before, with the virtual interface bound to the ALU's signals inside the subsystem. This is why checking must not depend on the stimulus components.

Q39. Why should the reference model not use the DUT's own package functions or internal signals? Any logic shared between DUT and model is not being verified, because both sides produce the same answer whether it is right or wrong. Sharing type definitions (the opcode enum) is fine; sharing behaviour is not.

Q40. How would you add functional coverage to this testbench? Add a coverage component with a covergroup sampled on each observed transaction. The monitor sends each transaction to a second mailbox for

the coverage component (or the scoreboard samples it). Coverage is taken from monitored transactions, not generated ones, so that it reflects what actually reached the DUT.

Q41. What is the weakness of ending the test on a transaction count, and how do you guard against it? If a transaction is dropped anywhere, the count is never reached and the simulation hangs. A watchdog timeout running in parallel converts the hang into an error.

Q42. The driver is killed by `disable fork` in the middle of driving. Is that a problem? Not in this testbench: `disable fork` is executed only after every transaction has been checked, so the driver is idle, blocked on the mailbox. In general, killing a driver mid-transaction leaves the bus in an undefined protocol state, which matters if the simulation continues afterwards (for example a reset test). Then the driver should be stopped cleanly and the bus returned to idle.

Q43. Why is `configure()` called from `run()` and not from the constructor? A constructor of a base class runs before the derived class's part of the object is initialised, and calling a virtual method from a constructor is a well-known source of surprises. Calling it at the start of `run()`, when the whole object tree exists, is safe and also guarantees that every component is already built when the settings are applied.

Q44. How would you test reset in the middle of operation with this architecture? A reset generator thread (driven by the test) asserts `rst_n` at a random time. The driver must abandon the current transaction and return to idle; the monitor already discards its pending transaction during reset; the scoreboard must be told to discard expected items for transactions that were in flight, and the end-of-test count must account for them.

Q45. What makes a testbench reusable? Components with a single responsibility; no test-specific code inside components; communication only through transactions and well-defined channels; all pin access behind a virtual interface; configuration through knobs and replaceable objects; and a checking path that works without the stimulus path.

7.3 Output-Prediction Questions

Each of these is a small, self-contained module about a mechanism the layered testbench depends on. Work out the output first.

P1.

```
module p1;
  class item;
    int v;
  endclass

  mailbox #(item) mbx = new();
  item it, got;

  initial begin
    it = new();
    for (int i = 1; i <= 3; i++) begin
      it.v = i;
      mbx.put(it);
    end
    repeat (3) begin
      mbx.get(got);
      $display("%0d", got.v);
    end
  end
endmodule
```

Answer: `3`, `3`, `3`. One object was created and its handle was put into the mailbox three times. All three handles refer to that object, whose `v` was last set to 3. Moving `it = new();` inside the loop gives 1, 2, 3. This is the "new object per transaction" rule.

P2.

```
module p2;
  initial begin
    fork
      begin #10 $display("A"); end
      begin #20 $display("B"); end
      begin #30 $display("C"); end
    join_any
    $display("after join_any at %0t", $time);
    #15;
    disable fork;
    $display("end at %0t", $time);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
A
after join_any at 10
B
end at 25
```

`join_any` returns when the first thread finishes (time 10). The other two keep running: B prints at 20. At 25 `disable fork` kills the remaining thread, so C never prints. This is the environment's start-up and shutdown in miniature.

P3.

```
module p3;
  class txn;
    int result;
  endclass

  txn actual, expected;

  initial begin
    actual          = new();
    actual.result   = 44;
    expected        = actual;
    expected.result = 300;
    $display("%0d %0d %s", actual.result, expected.result,
             (actual.result == expected.result) ? "PASS" : "FAIL");
  end
endmodule
```

Answer: `300 300 PASS`. `expected = actual;` copies the handle, not the object. Writing the prediction through `expected` overwrote the DUT's value, and the object was then compared with itself. A scoreboard written this way can never fail.

P4.

```
module p4;
  event e;

  initial begin
    -> e;
    $display("triggered at %0t", $time);
  end

  initial begin
    #1;
    @(e);
    $display("received at %0t", $time);
  end

  initial #10 $display("end at %0t", $time);
endmodule
```

Answer:

EXPECTED OUTPUT

```
triggered at 0
end at 10
```

The event is triggered at time 0, but the waiting process only reaches `@(e)` at time 1. A trigger is not remembered, so the second block waits forever and "received" is never printed. In a testbench this is a hung generator or a test that only ends by timeout.

P5.

```
module p5;
  initial begin
    for (int i = 0; i < 3; i++)
      fork
        $display("i=%0d", i);
      join_none
    #1 $display("done");
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
i=3
i=3
i=3
done
```

Threads created with `join_none` do not start until the parent blocks, which here is at `#1`, after the loop has finished and `i` is 3. All three threads share the one variable `i`. This is the bug that appears when a loop starts one driver per interface. The fix is a per-iteration copy: `automatic int k = i;` declared inside the `fork` before the statement, and `k` used in the thread.

P6.

```
module p6;
  mailbox #(int) m = new(2);

  initial begin
    fork
      for (int i = 1; i <= 4; i++) begin
        m.put(i);
        $display("[%0t] put %0d", $time, i);
      end
      begin
        int v;
        #10;
        repeat (4) begin
          m.get(v);
          $display("[%0t] got %0d", $time, v);
          #10;
        end
      end
    join
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
[0] put 1
[0] put 2
[10] got 1
[10] put 3
[20] got 2
[20] put 4
[30] got 3
[40] got 4
```

The mailbox is bounded to two entries. The producer puts 1 and 2 and then blocks in `put(3)`. Each `get()` frees one place, which lets exactly one more `put()` complete. A bounded mailbox paces the producer to the consumer's speed without any event. (The consumer continues to its `$display` before the unblocked producer runs, so "got" is printed before "put" at each time.)

P7.

```
module p7;
  class base_test;
    function void configure();
      $display("base configure");
    endfunction
    function void run();
      configure();
    endfunction
  endclass

  class my_test extends base_test;
    function void configure();
      $display("my configure");
    endfunction
  endclass

  base_test t;
  my_test   m;

  initial begin
    m = new();
    t = m;
    t.run();
    m.configure();
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
base configure
my configure
```

`configure()` is not virtual. Inside `base_test::run` the call is bound to `base_test::configure` at compile time, even though the object is a `my_test`. Called directly through the `my_test` handle, the derived version runs. With `virtual function void configure();` in the base class both lines would be `my configure`. A test whose settings are mysteriously ignored usually has this cause.

7.4 Find-the-Error and Debugging Questions

For each fragment, say what goes wrong and how to fix it.

D1.

```
class my_env;
  mailbox #(alu_txn) gen2drv_mbx;
  alu_generator      gen;
  alu_driver         drv;
  event              drv_done, gen_done;
  function new(virtual alu_if vif);
    gen = new(gen2drv_mbx, drv_done, gen_done);
    drv = new(vif, gen2drv_mbx, drv_done);
    gen2drv_mbx = new();
  endfunction
endclass
```

The mailbox is created after its (still null) handle has been passed to both components. Generator and driver hold `null`; the first `put()` is a null object access at time 0. Create the mailbox first.

D2.

```
function new(virtual alu_if vif);
  gen = new(new(), drv_done, gen_done);       // intention: "give it a mailbox"
  drv = new(vif, new(), drv_done);
endfunction
```

Even where a tool accepts this, each component receives its own, different mailbox. The generator fills one, the driver waits on the other, and the test hangs. One mailbox must be created, stored in a variable, and that same handle passed to both.

D3.

```
task run();
  alu_txn t;
  t = new();
  forever begin
    @(vif.mon_cb);
    if (vif.mon_cb.valid_out === 1'b1) begin
      t.result = vif.mon_cb.result;
      mon2scb_mbx.put(t);
    end
  end
endtask
```

Two faults. One object is reused for every transaction (move `t = new();` inside the `if`). And the monitor never samples the inputs, so the scoreboard has nothing to predict from.

D4.

```
task drive(alu_txn txn);
  @(posedge vif.clk);
  vif.valid_in = 1'b1;
  vif.a        = txn.a;
  vif.b        = txn.b;
  vif.op       = txn.op;
  @(posedge vif.clk);
  vif.valid_in = 1'b0;
endtask
```

Blocking assignments to DUT inputs at the active clock edge race with the DUT's `always_ff`. The DUT may see the new values at the same edge or one edge later, depending on the simulator. Drive through the clocking block (`vif.drv_cb.valid_in <= 1'b1;` after `@(vif.drv_cb)`). In this project there is a second fault: these signals are clocking block outputs, and mixing direct assignments with clocking drives on the same variable is asking for trouble.

D5.

```
task run();
  fork
    gen.run();
    drv.run();
    mon.run();
    scb.run();
  join
  scb.report();
endtask
```

`join` waits for all four threads; three are endless. `report()` is never reached. Use `join_any`, then drain, then `disable fork`.

D6.

```
task run();
  fork
    gen.run();
    drv.run();
    mon.run();
    scb.run();
  join_any
  scb.report();
  $finish;
endtask
```

The report and `$finish` happen at the instant the generator finishes. The last transaction has not reached the scoreboard yet. Wait for `scb.num_checked == gen.num_sent` (with a timeout) first.

D7.

```
function void check_txn(alu_txn actual);
  logic [8:0] exp_result;
  exp_result = alu_predict(actual);
  if (actual.result != exp_result) begin
    num_fail++;
    $error("mismatch");
  end
endfunction
```

Three faults. `!=` yields X when `result` contains X, so an X output is not reported: use `!==`. The message contains none of the information needed to debug. And nothing counts checked transactions, so a run in which nothing arrived looks the same as a clean run.

D8.

```
virtual task run();
  repeat (num_txns) begin
    blueprint.randomize();
    gen2drv_mbx.put(blueprint);
  end
endtask
```

The return value of `randomize()` is ignored, so a constraint failure goes unnoticed. The blueprint itself is put into the mailbox each time, so all entries are the same object and (with no pacing here) the driver sees only the last values. Check the return value and send `blueprint.copy()`.

D9.

```
class alu_monitor;
  virtual alu_if     vif;
  mailbox #(alu_txn) mon2scb_mbx;
  function new(mailbox #(alu_txn) mon2scb_mbx);
    this.mon2scb_mbx = mon2scb_mbx;
  endfunction
  ...
endclass
```

`vif` is never assigned. The first `@(vif.mon_cb)` dereferences a null virtual interface: a fatal run-time error at time 0. Pass the interface through the constructor.

D10.

```
function logic [8:0] alu_predict(alu_txn t);
  logic [7:0] r;
  case (t.op)
    ALU_ADD: r = t.a + t.b;
    ...
  endcase
  return r;
endfunction
```

`r` is 8 bits wide, so the addition is done in 8 bits and the carry is lost; the returned value has bit 8 always 0. Every addition that overflows is reported as a DUT failure although the DUT is right. Declare `r` as `logic [8:0]`.

D11.

```
// tb_top
initial begin
  alu_test test;
  test.run();
  $finish;
end
```

The test object is never constructed. `test` is `null` and `test.run()` is a null object access. Add `test = new(aif);`.

D12.

```
// In the generator
task send(alu_txn txn);
  gen2drv_mbx.put(txn);
  #1;
  @(drv_done);
endtask
```

Not a compile error, and it works with this driver, but it is fragile: adding a delay between `put()` and the wait opens a window in which a faster driver could trigger `drv_done` before the generator is waiting, and the generator would hang. Wait immediately after the `put()`, or use a mechanism that remembers (`wait (drv_done.triggered)` in the same time step, a bounded mailbox, or a semaphore).

D13. A colleague's test log ends with:

```
SCOREBOARD SUMMARY: checked=0 pass=0 fail=0
TEST PASSED
```

The report treats "no failures" as a pass even though nothing was checked. The stimulus never reached the scoreboard (reset never released, monitor not sampling, wrong mailbox). The report must fail when `num_checked` is 0, and the cause must be found.

D14. The compile fails with "class `alu_txn` is declared more than once" or, in another tool, with type-mismatch errors between `alu_txn` and `alu_tb_pkg::alu_txn`.

The class files were compiled both directly (listed on the command line) and through the package's `` `include ``. Compile only the package.

7.5 Coding Exercises with Solutions

Exercise 1: a miniature layered flow with no DUT

Build the smallest structure that has the shape of the real one: a producer that sends three items through a mailbox, a middle stage that stands in for "driver + DUT + monitor" (it takes 3 time units and doubles the value), and a checker that verifies the doubling. Start the three with `fork`...`join_any` and end cleanly.

ex01_mini_layered.sv

```
module ex01_mini_layered;

  class item;
    int unsigned id;
    bit [7:0]    din;
    bit [8:0]    dout;
  endclass

  class producer;
    mailbox #(item) out_mbx;
    int unsigned    count = 3;

    function new(mailbox #(item) out_mbx);
      this.out_mbx = out_mbx;
    endfunction

    task run();
      item it;
      for (int unsigned i = 1; i <= count; i++) begin
        it     = new();                  // a new object for every item
        it.id  = i;
        it.din = 50 * i;
        out_mbx.put(it);
        $display("[%0t] PRODUCER sent id=%0d din=%0d", $time, it.id, it.din);
        #10;
      end
    endtask
  endclass

  class doubler;                          // stands in for driver + DUT + monitor
    mailbox #(item) in_mbx;
    mailbox #(item) out_mbx;

    function new(mailbox #(item) in_mbx, mailbox #(item) out_mbx);
      this.in_mbx  = in_mbx;
      this.out_mbx = out_mbx;
    endfunction

    task run();
      item it;
      forever begin
        in_mbx.get(it);
        #3;
        it.dout = it.din * 2;
        out_mbx.put(it);
      end
    endtask
  endclass

  class result_checker;
    mailbox #(item) in_mbx;
    int unsigned    num_checked;
    int unsigned    num_fail;

    function new(mailbox #(item) in_mbx);
      this.in_mbx = in_mbx;
    endfunction

    task run();
```

```
      item      it;
      bit [8:0] expected;
      forever begin
        in_mbx.get(it);
        expected = it.din * 2;
        if (it.dout !== expected) begin
          num_fail++;
          $display("[%0t] CHECKER id=%0d din=%0d dout=%0d FAIL, expected %0d",
                   $time, it.id, it.din, it.dout, expected);
        end
        else
          $display("[%0t] CHECKER id=%0d din=%0d dout=%0d PASS",
                   $time, it.id, it.din, it.dout);
        num_checked++;
      end
    endtask
  endclass

  mailbox #(item) p2d_mbx = new();
  mailbox #(item) d2c_mbx = new();
  producer        prod;
  doubler         dbl;
  result_checker  chk;

  initial begin
    prod = new(p2d_mbx);
    dbl  = new(p2d_mbx, d2c_mbx);
    chk  = new(d2c_mbx);

    fork
      prod.run();
      dbl.run();
      chk.run();
    join_any
    wait (chk.num_checked == prod.count);
    disable fork;
    $display("[%0t] checked=%0d fail=%0d", $time, chk.num_checked, chk.num_fail);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] PRODUCER sent id=1 din=50
[3] CHECKER id=1 din=50 dout=100 PASS
[10] PRODUCER sent id=2 din=100
[13] CHECKER id=2 din=100 dout=200 PASS
[20] PRODUCER sent id=3 din=150
[23] CHECKER id=3 din=150 dout=300 PASS
[30] checked=3 fail=0
```

The producer finishes at time 30 (its last `#10`), which releases `join_any`. All three items have already been checked, so the `wait` passes immediately and `disable fork` stops the two endless threads. The class is named `result_checker` because `checker` is a reserved word in SystemVerilog.

Exercise 2: a queue-based reference model for a FIFO

Write the checking core of a FIFO scoreboard as a function that is called once per observed operation, and exercise it with a hand-written sequence that includes one wrong read value and one read from an empty FIFO.

ex02_fifo_model.sv

```
module ex02_fifo_model;

  typedef enum bit {FIFO_WRITE, FIFO_READ} fifo_kind_e;

  bit [7:0]    model_q [$];
  int unsigned num_checked;
  int unsigned num_fail;

  function automatic void observe(fifo_kind_e kind, logic [7:0] value);
    bit [7:0] expected_data;
    if (kind == FIFO_WRITE) begin
      model_q.push_back(value);
      $display("WRITE %h  model holds %0d", value, model_q.size());
    end
    else begin
      num_checked++;
      if (model_q.size() == 0) begin
        num_fail++;
        $display("READ  %h  FAIL: model is empty", value);
      end
      else begin
        expected_data = model_q.pop_front();
        if (value !== expected_data) begin
          num_fail++;
          $display("READ  %h  FAIL: expected %h", value, expected_data);
        end
        else
          $display("READ  %h  PASS", value);
      end
    end
  endfunction

  initial begin
    observe(FIFO_WRITE, 8'hA1);
    observe(FIFO_WRITE, 8'hB2);
    observe(FIFO_READ,  8'hA1);
    observe(FIFO_WRITE, 8'hC3);
    observe(FIFO_READ,  8'hB2);
    observe(FIFO_READ,  8'hFF);     // the "DUT" returned FF instead of C3
    observe(FIFO_READ,  8'h00);     // the "DUT" produced data from an empty FIFO
    $display("checked=%0d fail=%0d left in model=%0d",
             num_checked, num_fail, model_q.size());
  end

endmodule
```

EXPECTED OUTPUT

```
WRITE a1  model holds 1
WRITE b2  model holds 2
READ  a1  PASS
WRITE c3  model holds 2
READ  b2  PASS
READ  ff  FAIL: expected c3
READ  00  FAIL: model is empty
checked=4 fail=2 left in model=0
```

The queue is the reference model. Order is preserved by `push_back` and `pop_front`. Note that after the wrong read the model and the DUT are back in step, because the model popped the entry it expected.

Exercise 3: add functional coverage

Add a coverage component to the ALU testbench that records which operations and which operand corner values were observed. (Covergroups are the subject of Module 10; this exercise shows where coverage plugs into the architecture.)

```
// alu_coverage.sv  (include it in alu_tb_pkg after alu_txn.sv)
class alu_coverage;

  mailbox #(alu_txn) mon2cov_mbx;
  alu_txn            txn;

  covergroup alu_cg;
    cp_op : coverpoint txn.op;
    cp_a  : coverpoint txn.a {
      bins zero   = {8'h00};
      bins max    = {8'hFF};
      bins others = {[8'h01:8'hFE]};
    }
    cp_b  : coverpoint txn.b {
      bins zero   = {8'h00};
      bins max    = {8'hFF};
      bins others = {[8'h01:8'hFE]};
    }
    op_x_a : cross cp_op, cp_a;
  endgroup

  function new(mailbox #(alu_txn) mon2cov_mbx);
    this.mon2cov_mbx = mon2cov_mbx;
    alu_cg           = new();
  endfunction

  task run();
    forever begin
      mon2cov_mbx.get(txn);
      alu_cg.sample();
    end
  endtask

  function void report();
    $display("Functional coverage = %0.1f%%", alu_cg.get_coverage());
  endfunction

endclass
```

Changes elsewhere:

File	Change
`alu_monitor.sv`	A second mailbox handle `mon2cov_mbx` (constructor argument); after `mon2scb_mbx.put(pending);` add `mon2cov_mbx.put(pending);`
`alu_env.sv`	Declare and create `mon2cov_mbx` and `alu_coverage cov;` ; pass the mailbox to the monitor and to `cov` ; add `cov.run();` to the first `fork`
`alu_test.sv`	After `env.scb.report();` add `env.cov.report();`

The same handle is put into both mailboxes. That is safe because neither the scoreboard nor the coverage component modifies the monitor's object. Coverage is sampled from **observed** transactions, so it measures what the DUT actually received. Generator, driver, scoreboard and all tests are untouched.

Exercise 4: a knob for idle cycles between transactions

Give the driver a setting `idle_cycles` (default 0) that inserts that many extra idle clock cycles after each transaction, and write a test that uses it.

```
// In alu_driver: one new property and two new lines in run()
int unsigned idle_cycles = 0;

task run();
  alu_txn txn;
  reset();
  forever begin
    gen2drv_mbx.get(txn);
    drive(txn);
    num_driven++;
    txn.print("DRV");
    repeat (idle_cycles) @(vif.drv_cb);       // extra idle cycles, bus already idle
    -> drv_done;
  end
endtask
```

```
// alu_slow_test.sv
class alu_slow_test extends alu_test;

  function new(virtual alu_if vif);
    super.new(vif);
    name = "alu_slow_test";
  endfunction

  virtual function void configure();
    env.gen.num_txns    = 5;
    env.drv.idle_cycles = 2;
  endfunction

endclass
```

With `idle_cycles` = 2 each transaction takes four clock cycles instead of two: the `DRV` lines appear at 35, 75, 115 and so on (40 time units apart instead of 20), and each `MON` / `SCB` pair still follows its `DRV` line by one clock. The monitor and scoreboard need no change: the monitor reacts to `valid_in` and `valid_out`, however far apart the transactions are. The default of 0 keeps every existing test's behaviour.

Exercise 5: a corner-case test

Write a test that sends 30 transactions in which both operands are always one of `00`, `01`, `7F`, `80`, `FE`, `FF`.

```
// alu_corner_test.sv
class alu_corner_txn extends alu_txn;

  constraint corner_c {
    a inside {8'h00, 8'h01, 8'h7F, 8'h80, 8'hFE, 8'hFF};
    b inside {8'h00, 8'h01, 8'h7F, 8'h80, 8'hFE, 8'hFF};
  }

endclass

class alu_corner_test extends alu_test;

  function new(virtual alu_if vif);
    super.new(vif);
    name = "alu_corner_test";
  endfunction

  virtual function void configure();
    alu_corner_txn corner_txn;
    corner_txn        = new();
    env.gen.blueprint = corner_txn;
    env.gen.num_txns  = 30;
  endfunction

endclass
```

Then add `` `include "alu_corner_test.sv" `` to the package and a branch to the `case` in `tb_top`. The new constraint is combined with the inherited `operand_c`. The two do not conflict: `operand_c` is a distribution over all values from `00` to `FF`, and `corner_c` restricts the choice to six of them, each of which the distribution allows.

Exercise 6: read the number of transactions from the command line

Make the base test accept `+NUM=<n>`.

```
// Replacement for alu_test::configure
virtual function void configure();
  int unsigned n;
  if ($value$plusargs("NUM=%d", n)) env.gen.num_txns = n;
  else                              env.gen.num_txns = 10;
endfunction
```

`./simv +TEST=alu_test +NUM=1000` now runs a thousand random transactions with no recompilation. The environment's drain logic needs no change, because it compares against `gen.num_sent`, the number actually sent.

Further practice (no solutions given)

1. Change the environment to use a bounded mailbox of size 1 between generator and driver, and remove the `drv_done` handshake. Which component's code changes, and what must the driver do so that the generator is held back until the transaction has really been applied?
2. Rewrite the monitor to send two kinds of transaction (inputs observed, outputs observed) through two mailboxes, and move the pairing into the scoreboard using a queue of expected results. When is this structure better than the one used in this module?
3. Extend the driver so that it can send transactions on consecutive clock cycles with no idle cycle between them. Check, by reasoning through the timeline, that the monitor of Section 4.1 already handles this.
4. Add a mid-test reset: a thread in the test asserts `rst_n` for two cycles at a random time. List every component that must react and what it must do.
5. Introduce each of these bugs into the DUT, one at a time, and write down what the directed test's log shows: `ALU_SUB` computes `b - a`; `ALU_SHR` shifts by two; `valid_out` is delayed by two cycles instead of one; `result` is not cleared by reset.
6. Introduce each of these bugs into the testbench and observe the symptom: the monitor reuses one object; the scoreboard uses `expected = actual`; the environment creates two mailboxes for generator and driver; `configure()` is not virtual.
7. Write the complete layered testbench for a 4-bit up-counter with `enable` and synchronous `load`. Decide first what a transaction is, and where the reference model's state lives.
8. Sketch (boxes and arrows) the testbench for a DUT with two ALUs sharing one output bus. How many drivers, monitors and scoreboards are there?

7.6 Concepts You Must Understand Before Moving Forward

Do not start Module 10 until you can answer each of these without looking at the notes.

Architecture

- [ ] I can draw the layered testbench from memory and state the stimulus path and the checking path.

- [ ] I can give the one-sentence job of the generator, driver, monitor, scoreboard, environment, test and top module.
- [ ] I can list three advantages and two disadvantages of each of the direct and the layered testbench.
- [ ] I can say which components know the pin-level protocol and which never see a signal.

Connections

- [ ] I can explain how one mailbox handle ends up in two components, and what goes wrong if two mailboxes are created.
- [ ] I can trace the virtual interface from `tb_top` to the driver.
- [ ] I can explain why class files are included in a package and not compiled directly.
- [ ] I can explain build, connect and run, and where each happens.

Components

- [ ] I can explain why the generator sends a copy of the blueprint, and how the blueprint lets a test change constraints.
- [ ] I can explain why the driver waits for a clocking event before each drive and uses `<=` through the clocking block.
- [ ] I can explain why the monitor creates a new object per transaction.
- [ ] I can explain how the monitor aligns inputs and outputs across the one-cycle latency.
- [ ] I can explain why the scoreboard predicts from monitored inputs, why the model must be independent of the RTL, and why it compares with `===`.
- [ ] I can write a reference model that keeps the carry bit, and explain the `~` width trap.

Control

- [ ] I can explain why the environment uses `fork` ... `join_any`, what happens after it returns, and what `disable fork` kills.
- [ ] I can describe a robust end-of-test condition and why it needs a timeout.
- [ ] I can add a test by extending the base test, in three different ways (knob, blueprint, generator).
- [ ] Given a scoreboard failure message, I can decide whether to suspect the DUT, the reference model or the stimulus path.
- [ ] I can describe how the scoreboard changes for a FIFO.

7.7 What Comes Next

Module 10 adds the two things this testbench is still missing. **Functional coverage** answers "which operations and operand corners did the random tests actually hit?" using covergroups sampled on the monitor's transactions (Exercise 3 showed the plug-in point). **Assertions** check the protocol continuously at signal level, for example that `valid_out` follows `valid_in` by exactly one cycle and that outputs are never X when valid; they live in the interface, next to the signals they watch. Module 11 then shows how UVM packages this whole architecture into a standard library.